Attention:
Teaching Models Where to Look

Danna Gurari
University of Colorado Boulder
Spring 2026

CC BY NC SA

https://dannagurari.colorado.edu/course/neural-networks-and-deep-learning-spring-2026/

Review

- Last lecture:
  - Isolated successes: CNNs for vision and RNNs for language
  - The next challenge: describing visual scenes with natural language
  - Early vision-language pioneers: connecting pixels and words
  - Programming tutorial

- Assignments (Canvas):
  - Problem set 2 due Friday (answers to common questions on assignment page)
  - Final project: find a partner (our 3-stage process to assist you)

- Questions? (can also post these to Piazza or attend office hours)

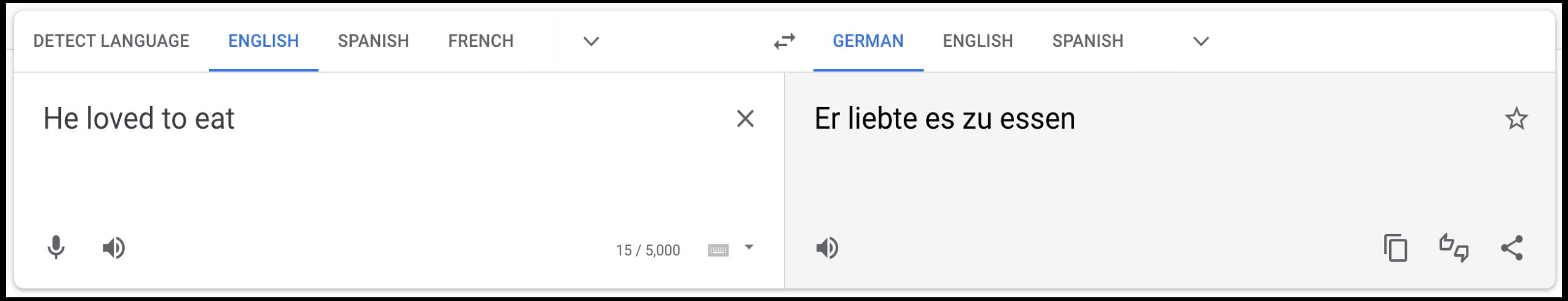
Today's Topics

- Mystery: why did neural networks struggle to translate long sentences?
- Idea: instead of summarizing everything, attend to relevant inputs
- Beyond text: attention for image captioning
- Beyond vision-language problems: attention for speech recognition

Today's Topics

- Mystery: why did neural networks struggle to translate long sentences?
- Idea: instead of summarizing everything, attend to relevant inputs
- Beyond text: attention for image captioning
- Beyond vision-language problems: attention for speech recognition

Motivating Task: Machine Translation



Which type of sequence problem is this: one-to-many, many-to-one, or many-to-many?

Recall: Pioneering Neural Network Approach

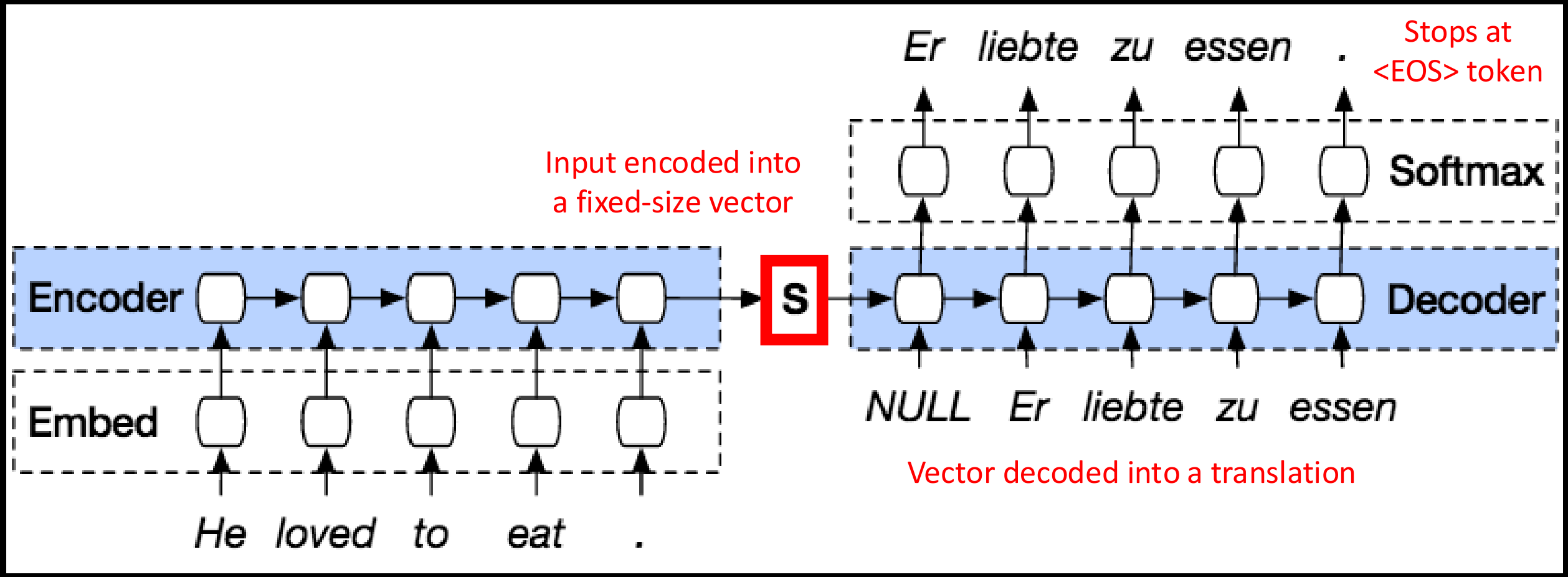
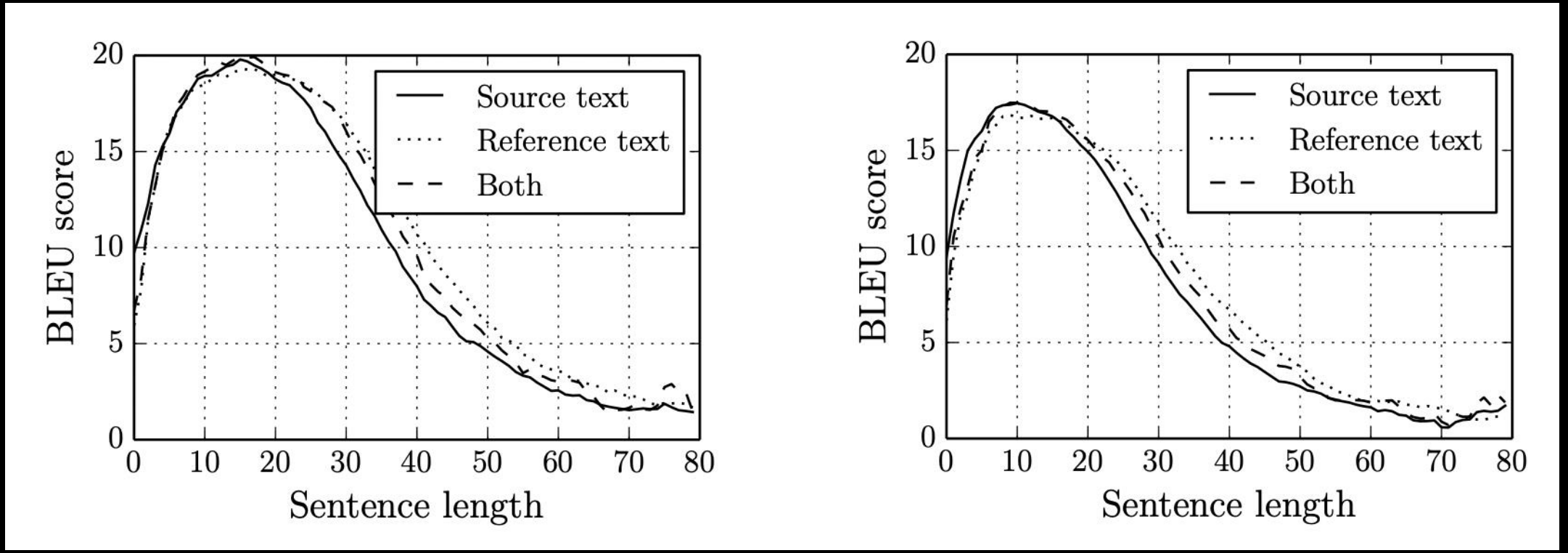


Image source: https://smerity.com/articles/2016/google_nmt_arch.html
seq2seq: Sutskever et al. Sequence to Sequence Learning with Neural Networks. Neurips 2014.

Analysis of Two Models

(larger scores are better)

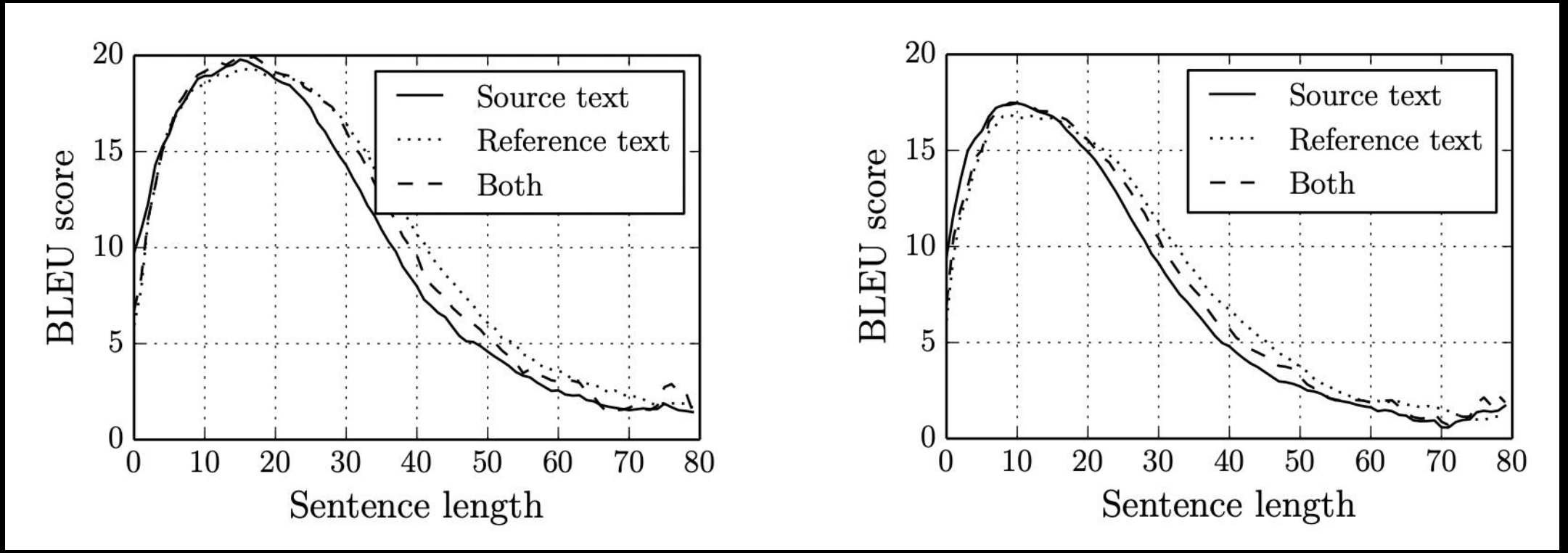


What performance trend is observed for inputs (source) and outputs (reference) as the number of words in each sentence grows?

Cho et al. On the Properties of Neural Machine Translation: Encoder–Decoder Approaches. SSST 2014

Analysis of Two Models

(larger scores are better)



Performance drops for longer sentences!

Cho et al. On the Properties of Neural Machine Translation: Encoder–Decoder Approaches. SSST 2014

Hypothesis: Insufficient Capacity in Fixed-Length Vector for All Relevant Information in Long Sentences

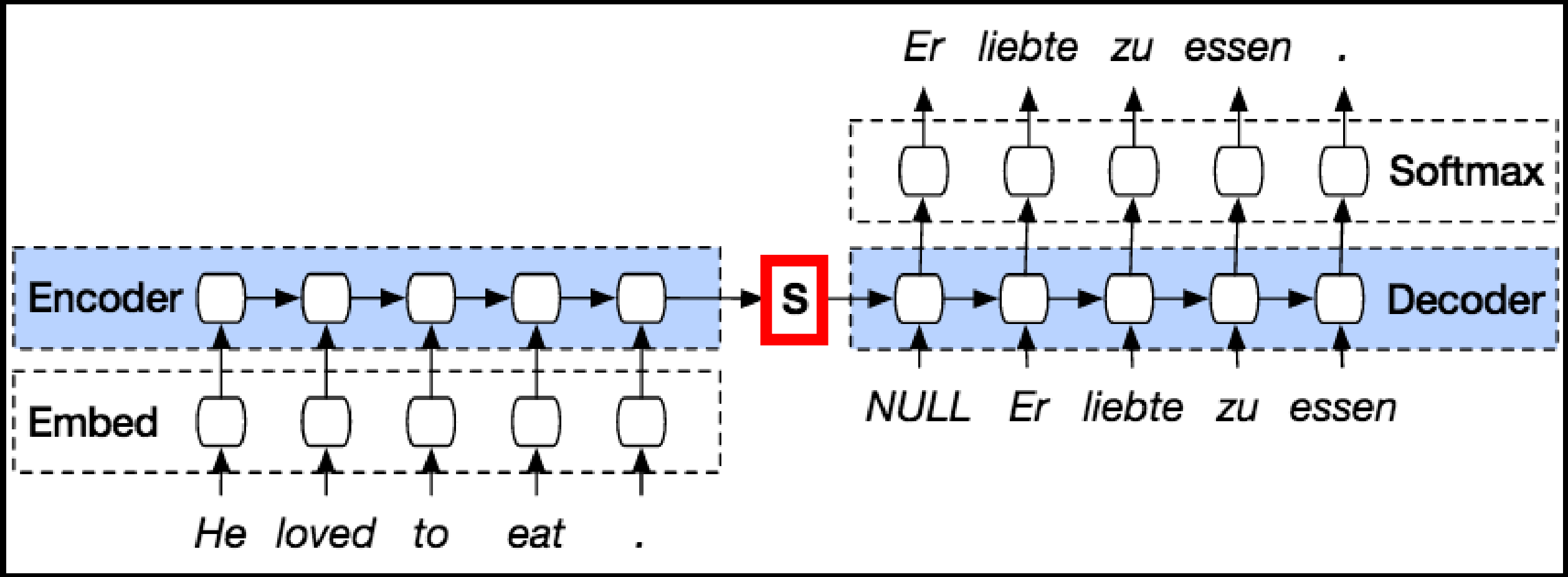


Image source: https://smerity.com/articles/2016/google_nmt_arch.html
seq2seq: Sutskever et al. Sequence to Sequence Learning with Neural Networks. Neurips 2014.

Today's Topics

- Mystery: why did neural networks struggle to translate long sentences?
- Idea: instead of summarizing everything, attend to relevant inputs
- Beyond text: attention for image captioning
- Beyond vision-language problems: attention for speech recognition

Published as a conference paper at ICLR 2015

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau
Jacobs University Bremen, Germany

KyungHyun Cho **Yoshua Bengio***
Université de Montréal

Idea to Preserve Performance for Long Sentences: **Attention**

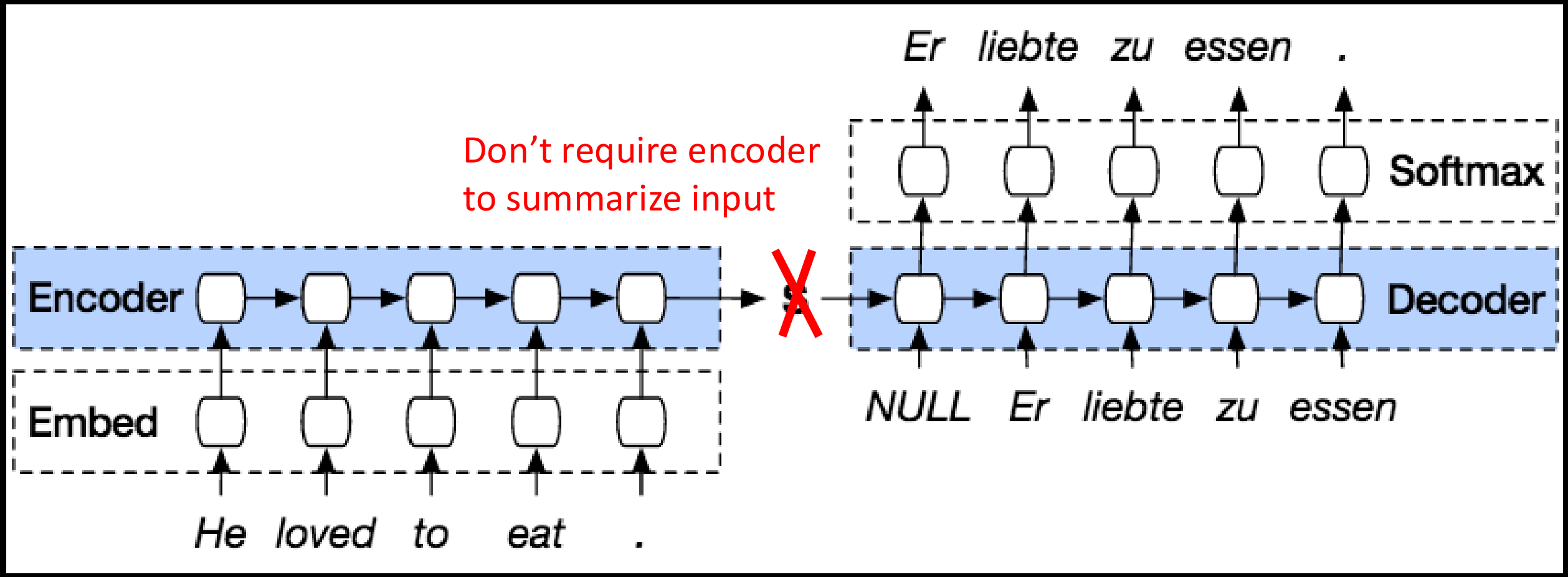


Image source: https://smerity.com/articles/2016/google_nmt_arch.html

Idea to Preserve Performance for Long Sentences: **Attention**

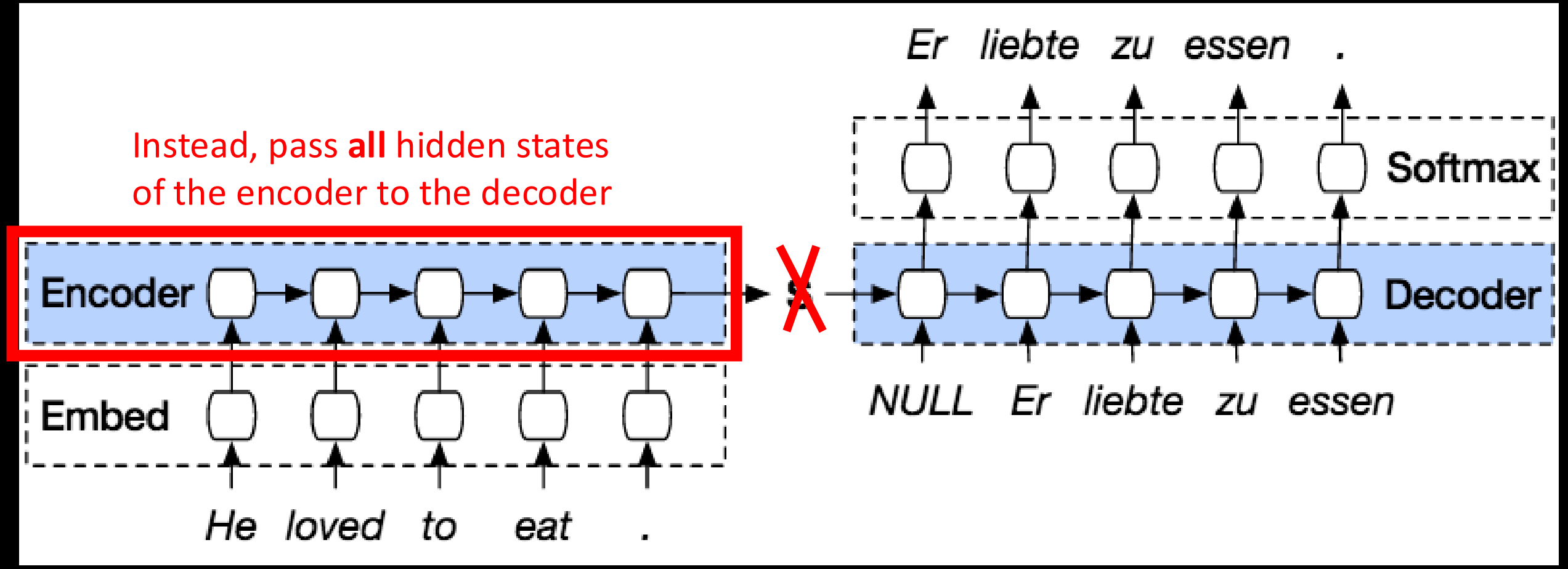
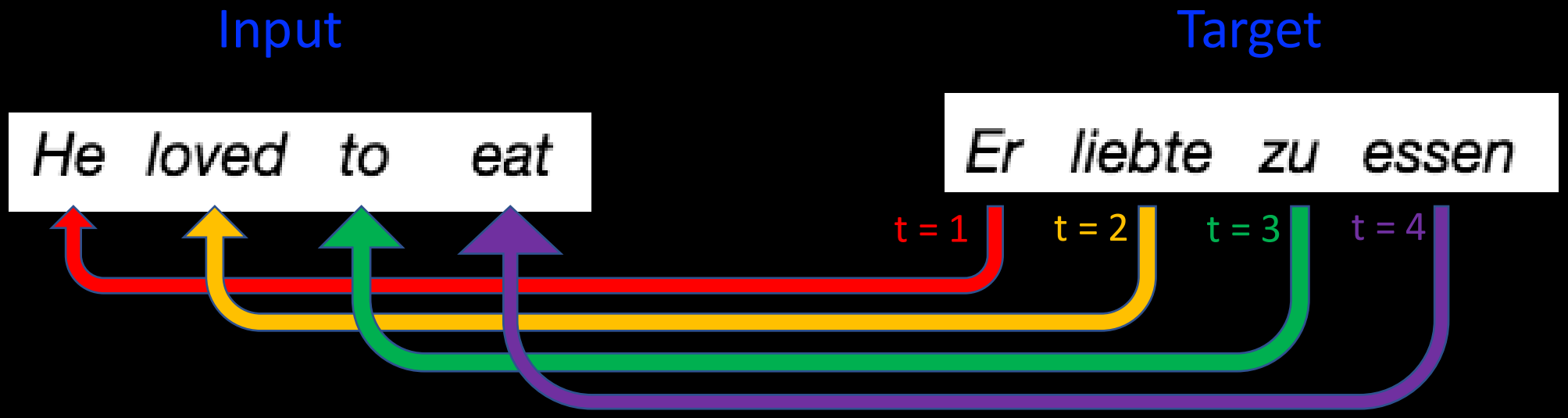


Image source: https://smerity.com/articles/2016/google_nmt_arch.html

Idea to Preserve Performance for Long Sentences: **Attention**

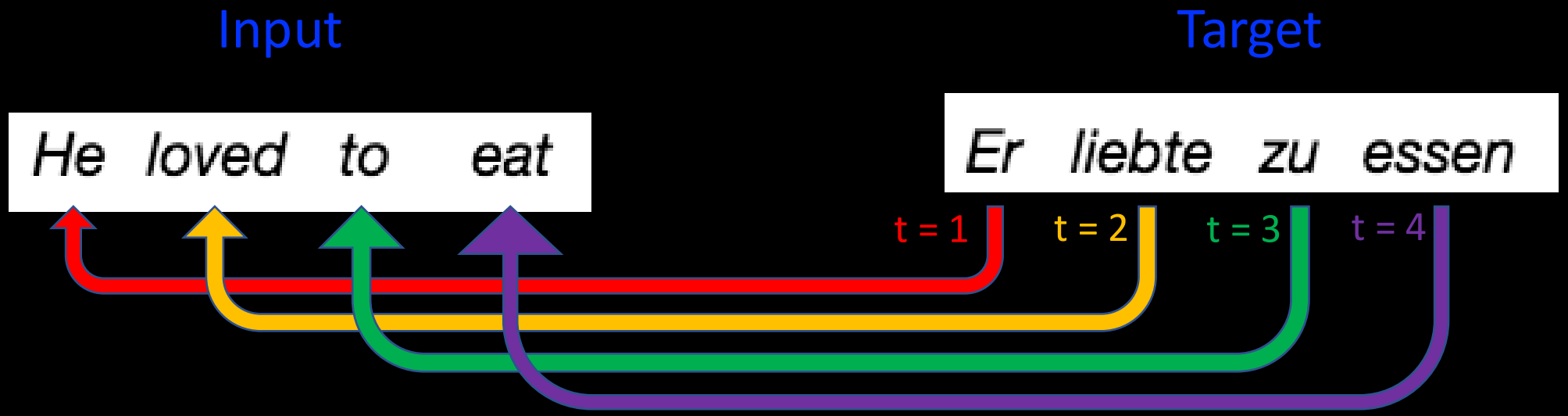
Decoder decides which inputs are needed for prediction at each time step; e.g., “hard attention” focuses on one input



Note: while word order between the input and target align in this example, it can differ

Idea to Preserve Performance for Long Sentences: **Attention**

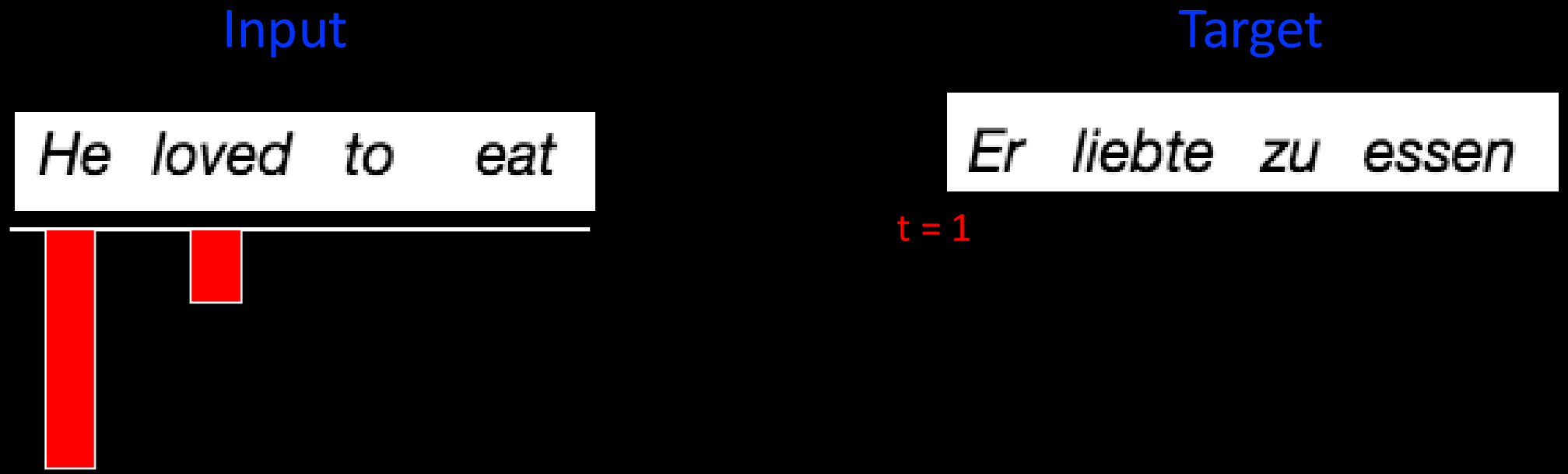
Decoder decides which inputs are needed for prediction at each time step; e.g., “hard attention” focuses on one input



Limitations: a target word relies on information about one input word and “hard attention” is not differentiable

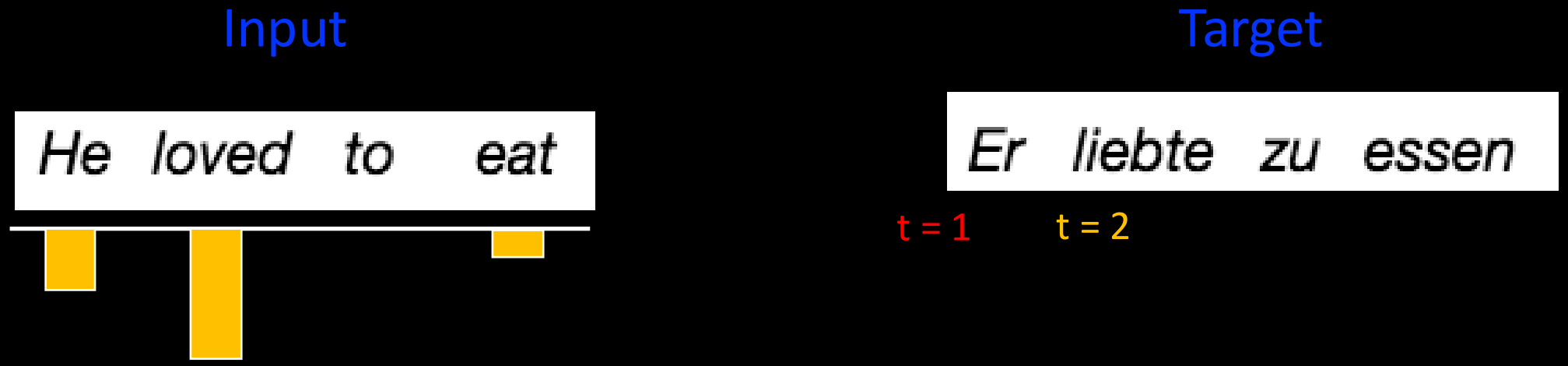
Idea to Preserve Performance for Long Sentences: **Attention**

Decoder decides which inputs are needed for prediction at each time step; e.g., “soft attention” uses a weighted combination of the input



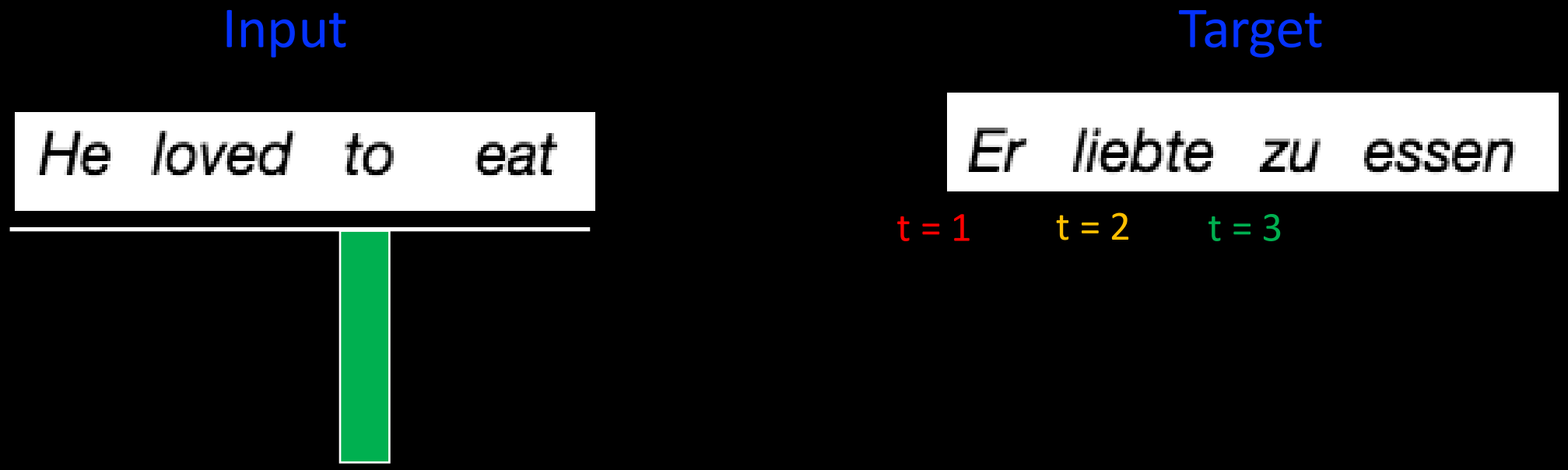
Idea to Preserve Performance for Long Sentences: **Attention**

Decoder decides which inputs are needed for prediction at each time step; e.g., “soft attention” uses a weighted combination of the input



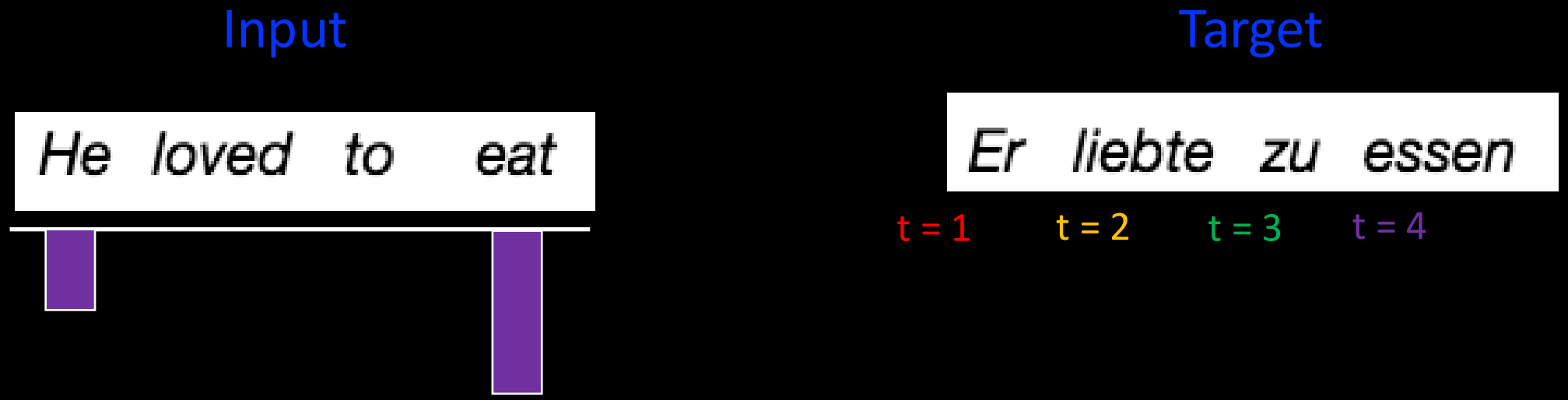
Idea to Preserve Performance for Long Sentences: **Attention**

Decoder decides which inputs are needed for prediction at each time step; e.g., “soft attention” uses a weighted combination of the input



Idea to Preserve Performance for Long Sentences: **Attention**

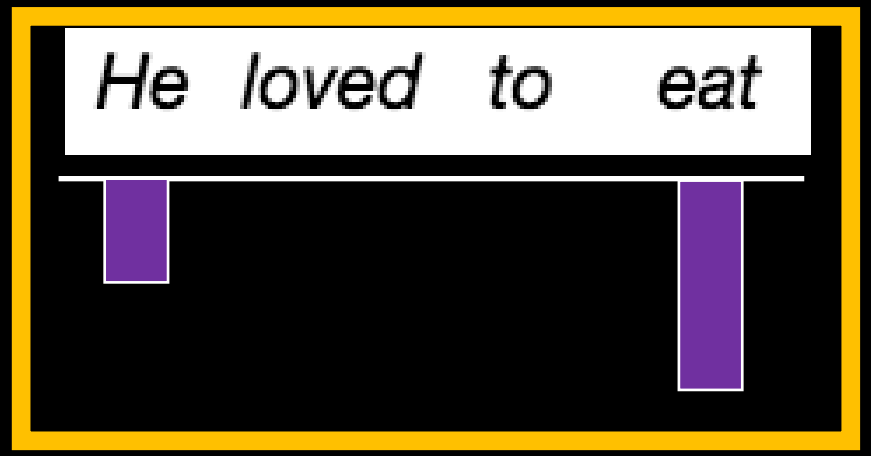
Decoder decides which inputs are needed for prediction at each time step; e.g., “soft attention” uses a weighted combination of the input



“Soft” Attention: Challenge

Decoder decides which inputs are needed for prediction at each time step; e.g., “soft attention” uses a weighted combination of the input

Input



Target

Er liebte zu essen

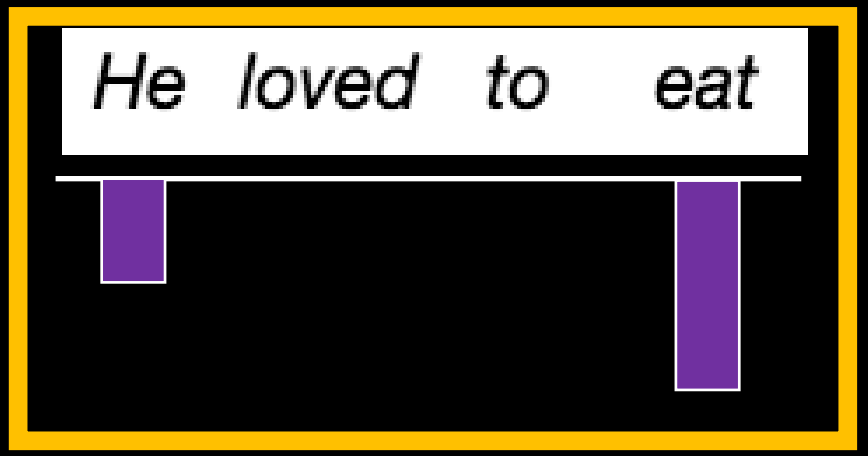
t = 1 t = 2 t = 3 t = 4

How should weights be chosen for each input?

“Soft” Attention: Challenge

Decoder decides which inputs are needed for prediction at each time step; e.g., “soft attention” uses a weighted combination of the input

Input



Target

Er liebte zu essen

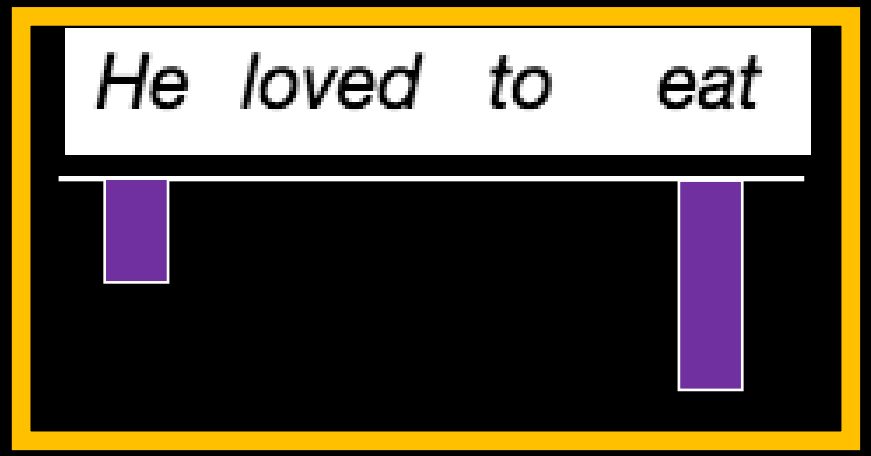
t = 1 t = 2 t = 3 t = 4

Collect manual annotations and then supervised learning to match ground truth parameters?
- This is impractical

“Soft” Attention: Challenge

Decoder decides which inputs are needed for prediction at each time step; e.g., “soft attention” uses a weighted combination of the input

Input



Instead, have the model learn how to weight each input!

Target

Er liebte zu essen

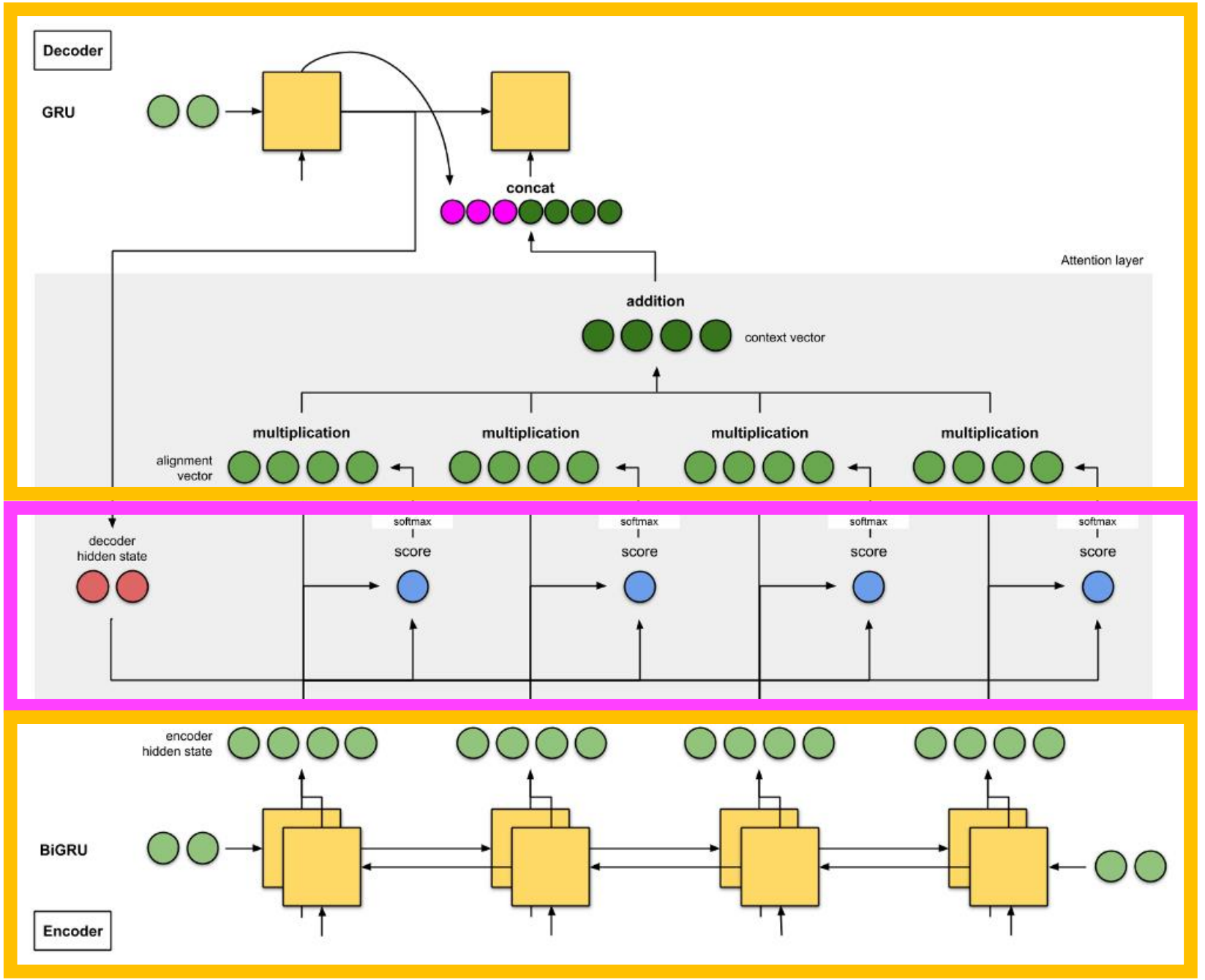
t = 1 t = 2 t = 3 t = 4

Idea: Modify Encoder-Decoder Architecture

Decoder: predicts word at each time step

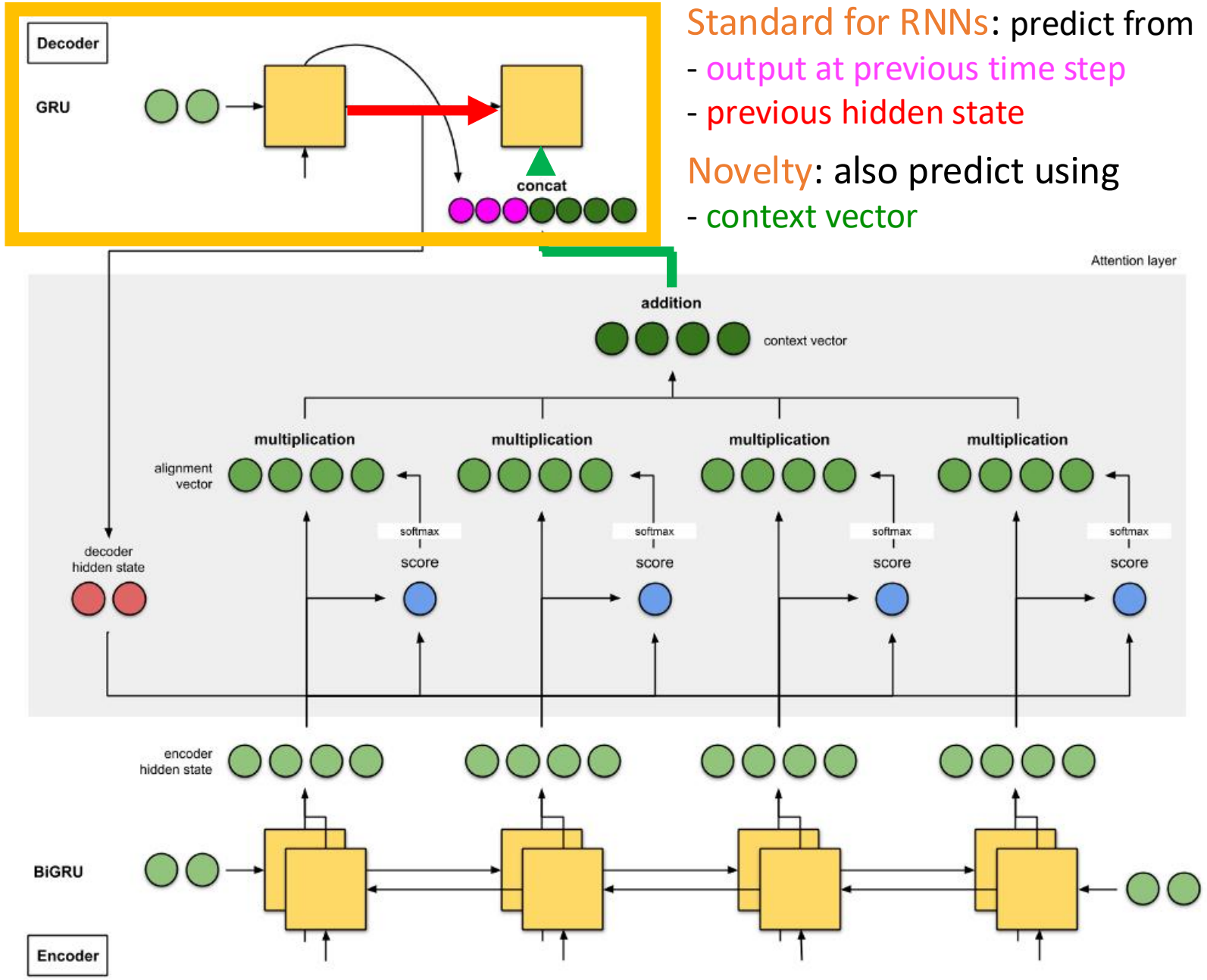
Attention module: determines each input's relevance for prediction

Encoder: converts each input into hidden states



https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Decoder: Word Prediction

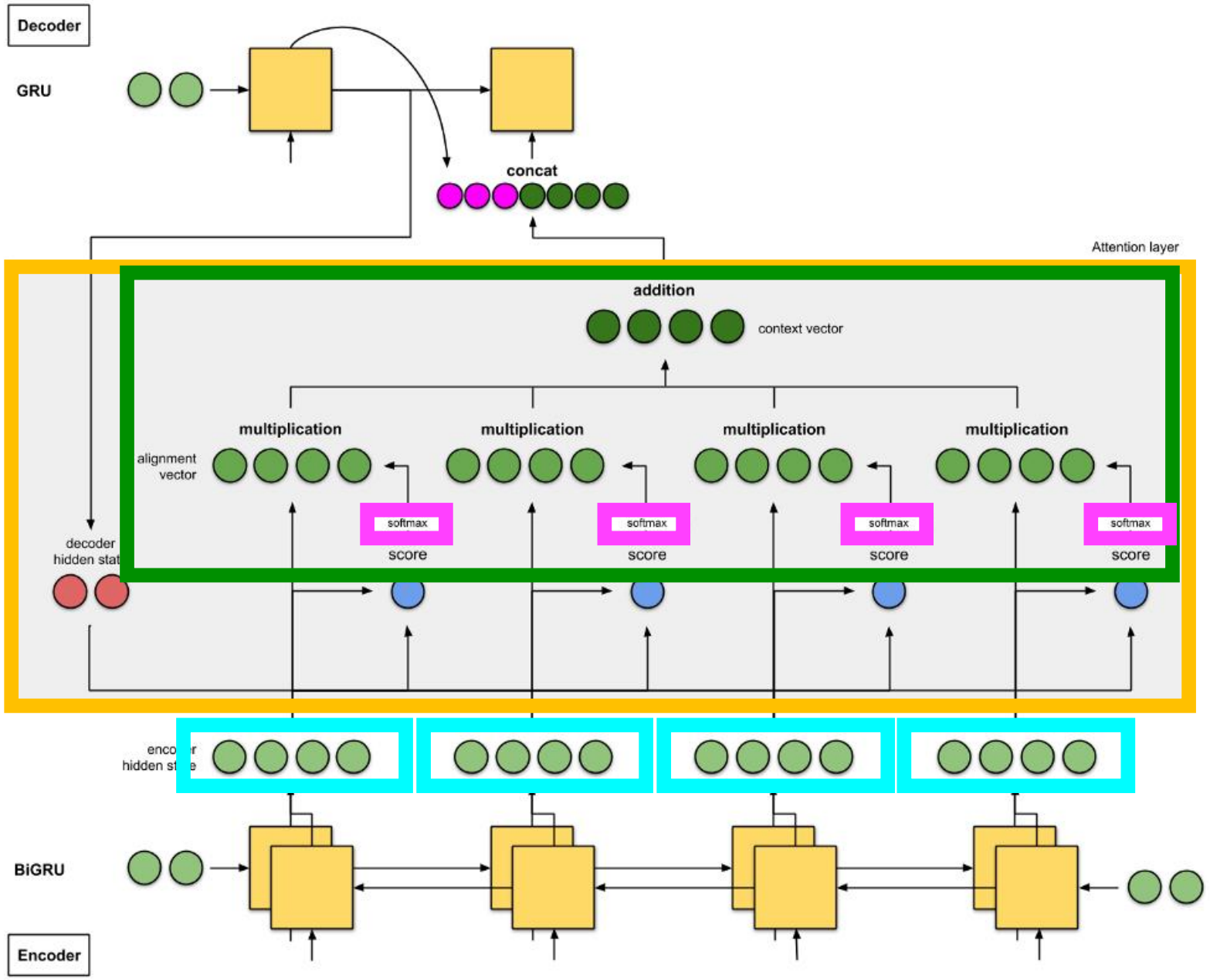


https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention: Creates Context Vector Used for Word Prediction

A context vector is just the weighted sum of all inputs at a given time step

How many inputs enter the attention layer in this example?



https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention: Creates Context Vector Used for Word Prediction

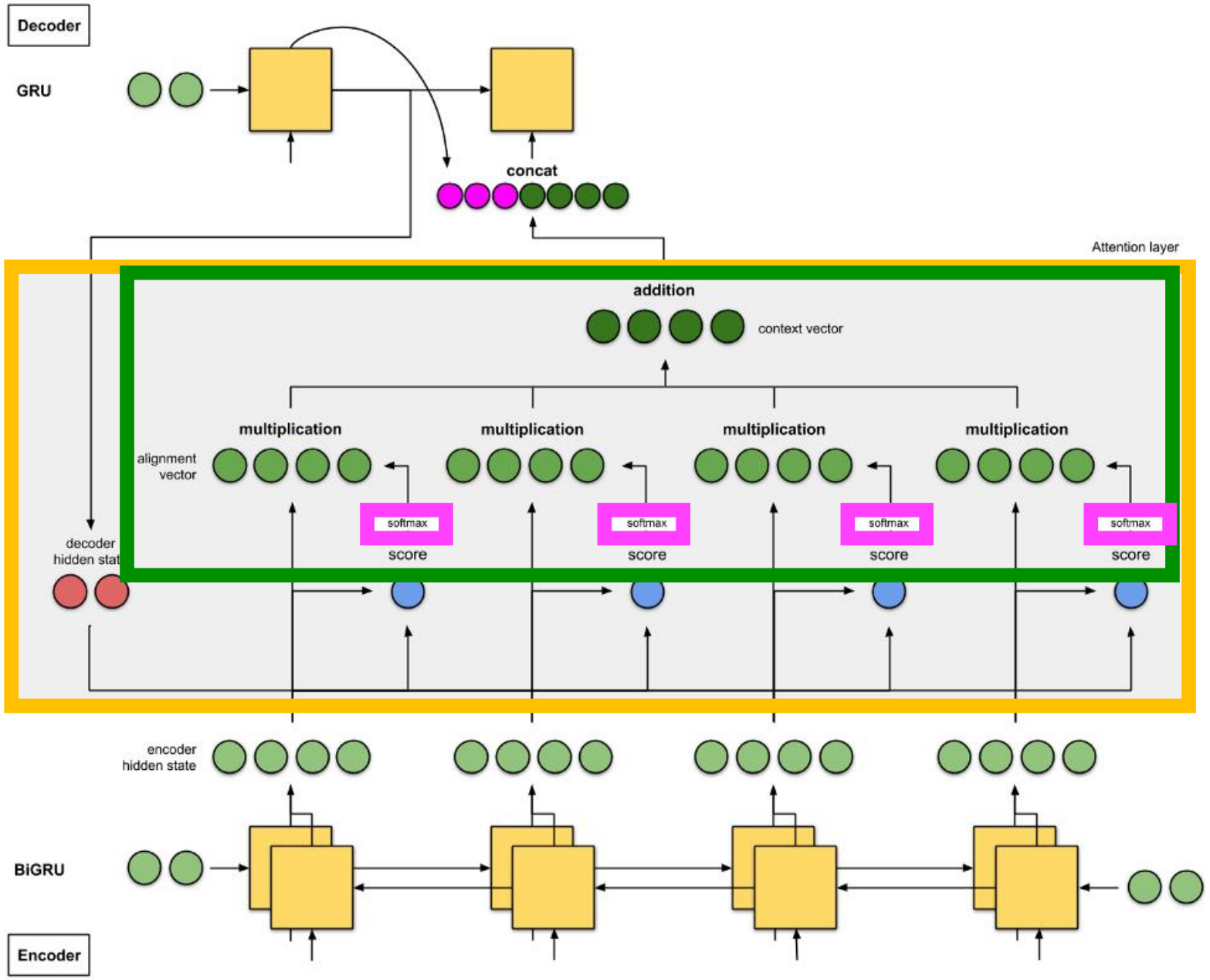
A context vector is just the weighted sum of all inputs at a given time step:

$$\mathbf{c}_t = \sum_{i=1}^{n} \alpha_{t,i} \boldsymbol{h}_i$$

What is the sum of all 4 weights?

What is the impact of a larger attention weight on an input's influence on a prediction:

- Amplification?
- Repression?

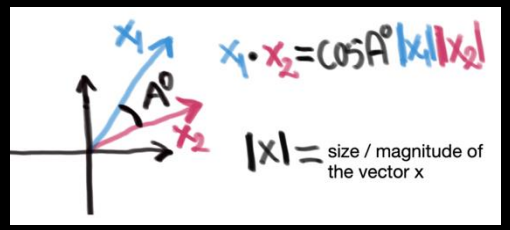


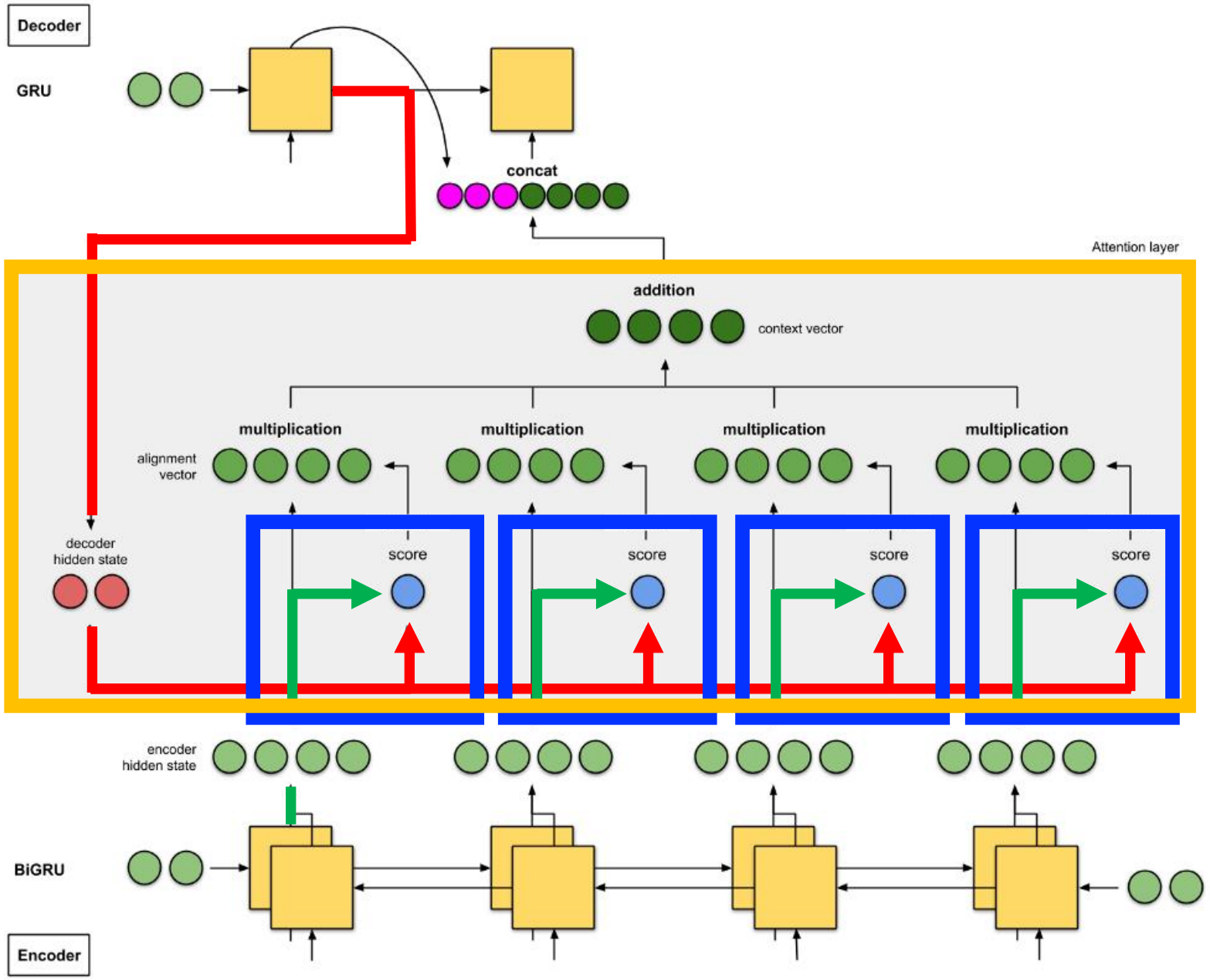
https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention: Computing an Input's Influence

For each input at each time step, compare the hidden states of the decoder ("what I am looking for") and input ("what I have")

e.g., dot product (uses 2 vectors)
- Match = 1 (i.e., cos(0))





https://towardsdatascience.com/self-attention-5b95ea164f61

https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention: Computing an Input's Influence

For each input at each time step, compare the hidden states of the decoder ("what I am looking for") and input ("what I have")

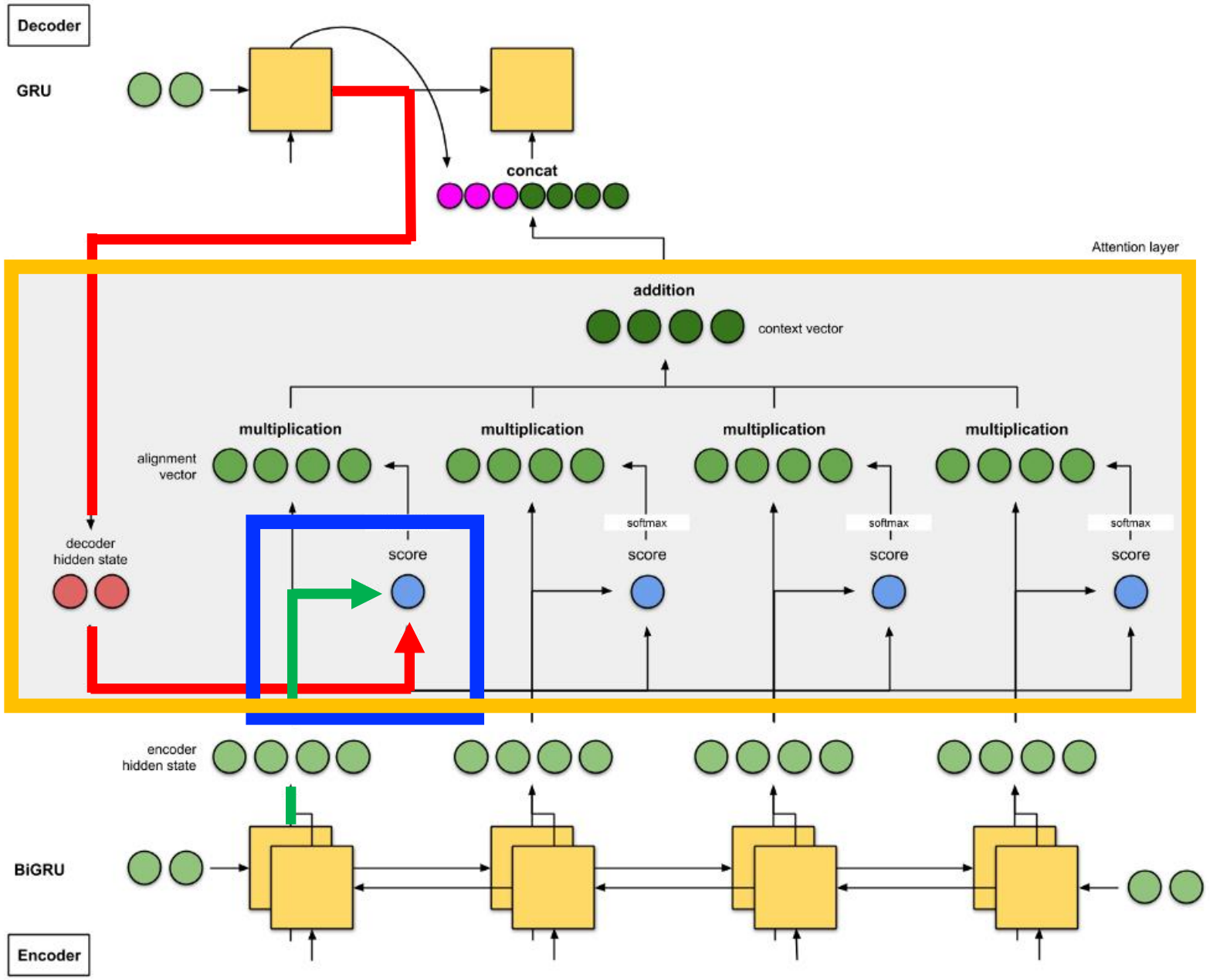
e.g., dot product (uses 2 vectors)

1	0	0	2

x

0
0
1
1

= ?



https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention: Computing an Input's Influence

For each input at each time step, compare the hidden states of the decoder ("what I am looking for") and input ("what I have")

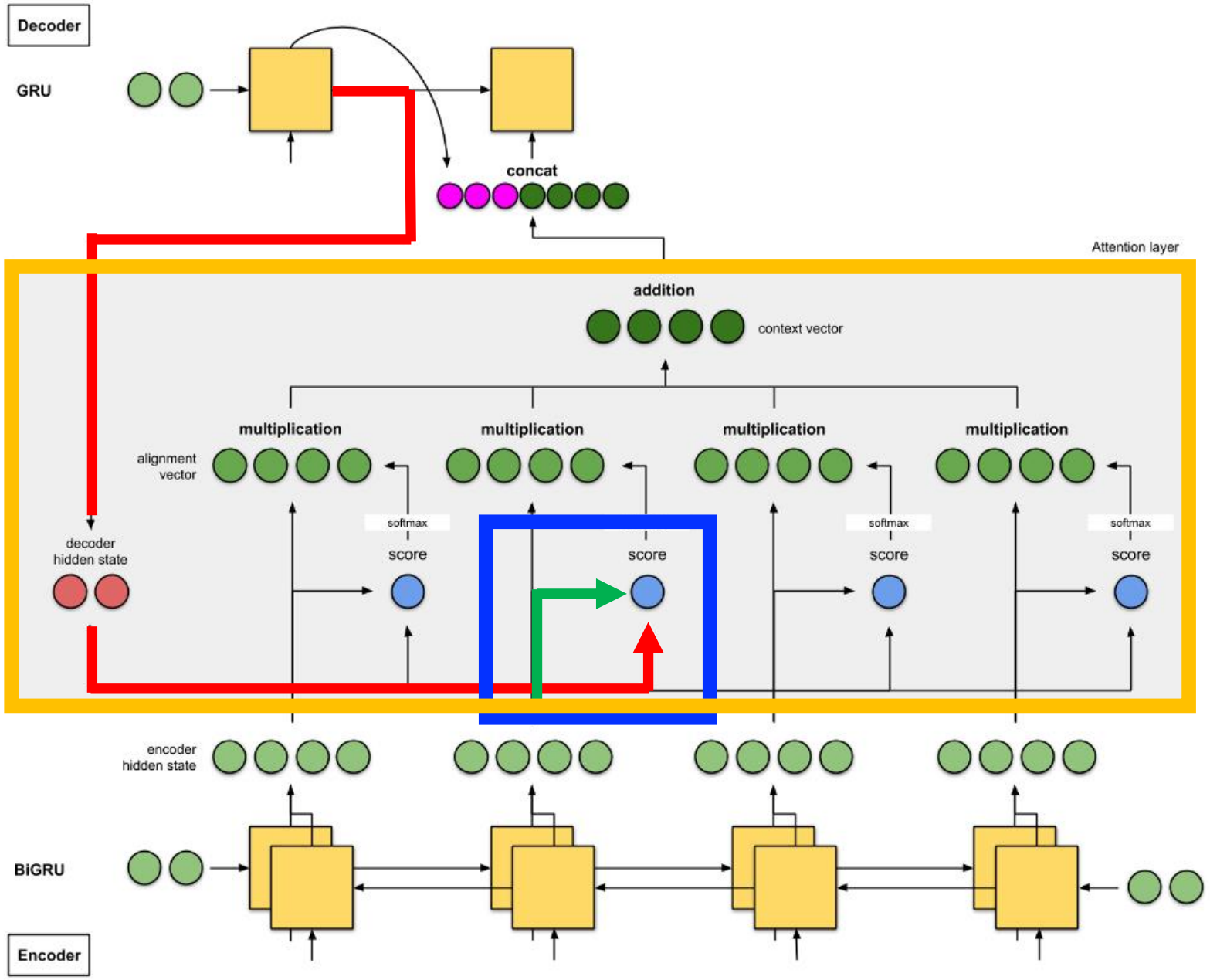
e.g., dot product (uses 2 vectors)

1	0	0	2

x

0
1
0
0

= ?



https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention: Computing an Input's Influence

For each input at each time step, compare the hidden states of the decoder ("what I am looking for") and input ("what I have")

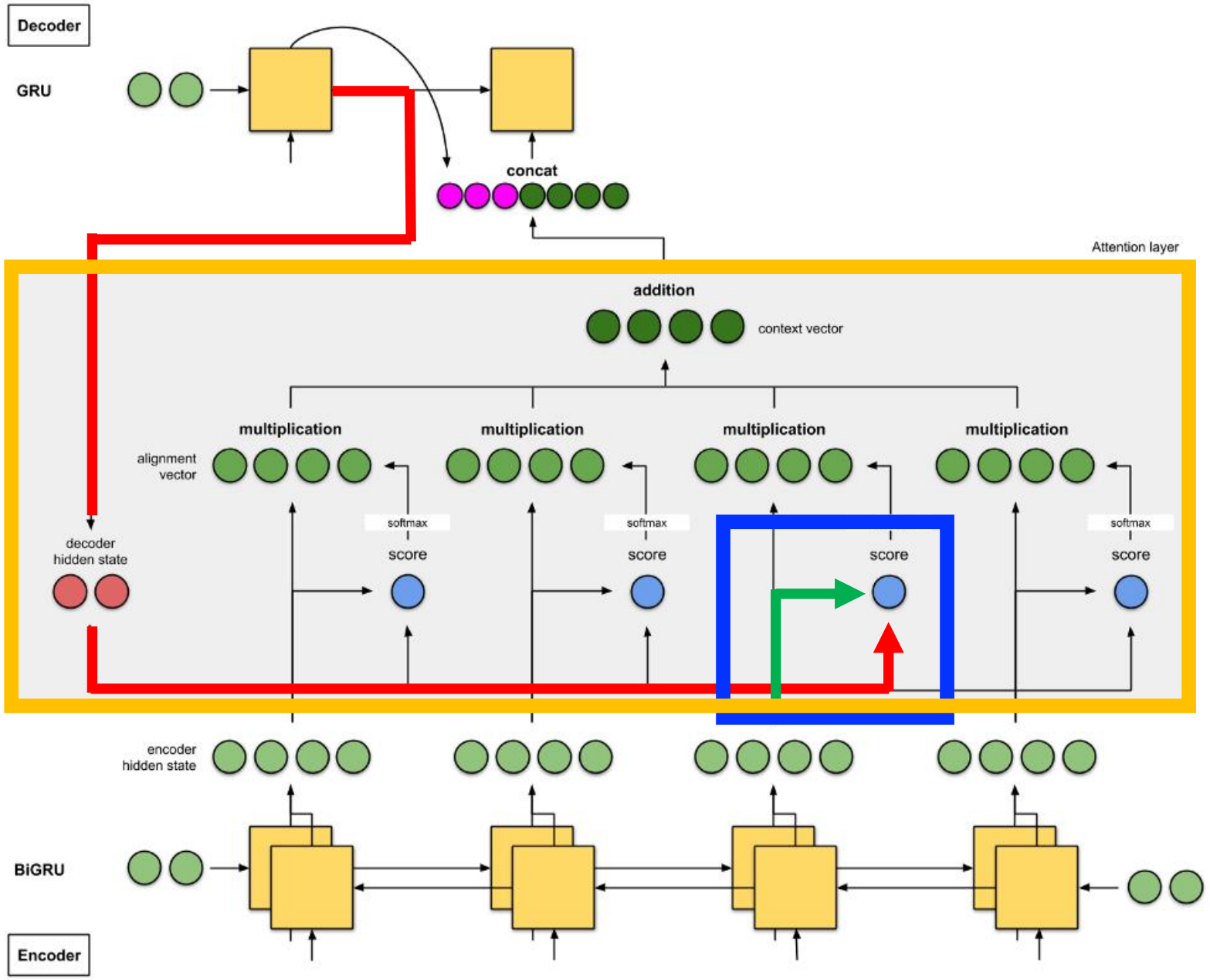
e.g., dot product (uses 2 vectors)

1	0	0	2

x

1
1
0
1

= ?



https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention: Computing an Input's Influence

For each input at each time step, compare the hidden states of the decoder ("what I am looking for") and input ("what I have")

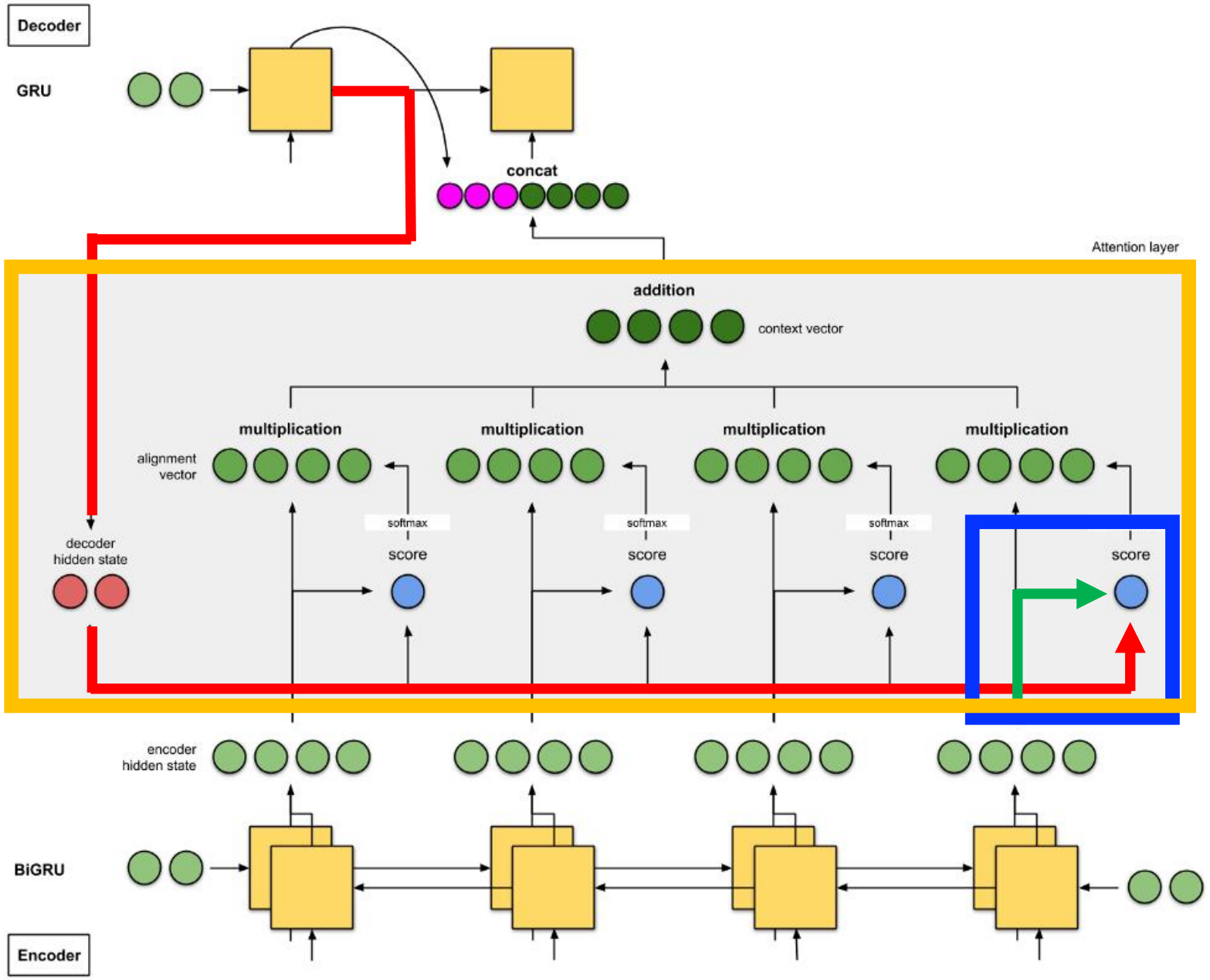
e.g., dot product (uses 2 vectors)

1	0	0	2

x

1
0
0
1

= ?



https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention: Computing an Input's Influence

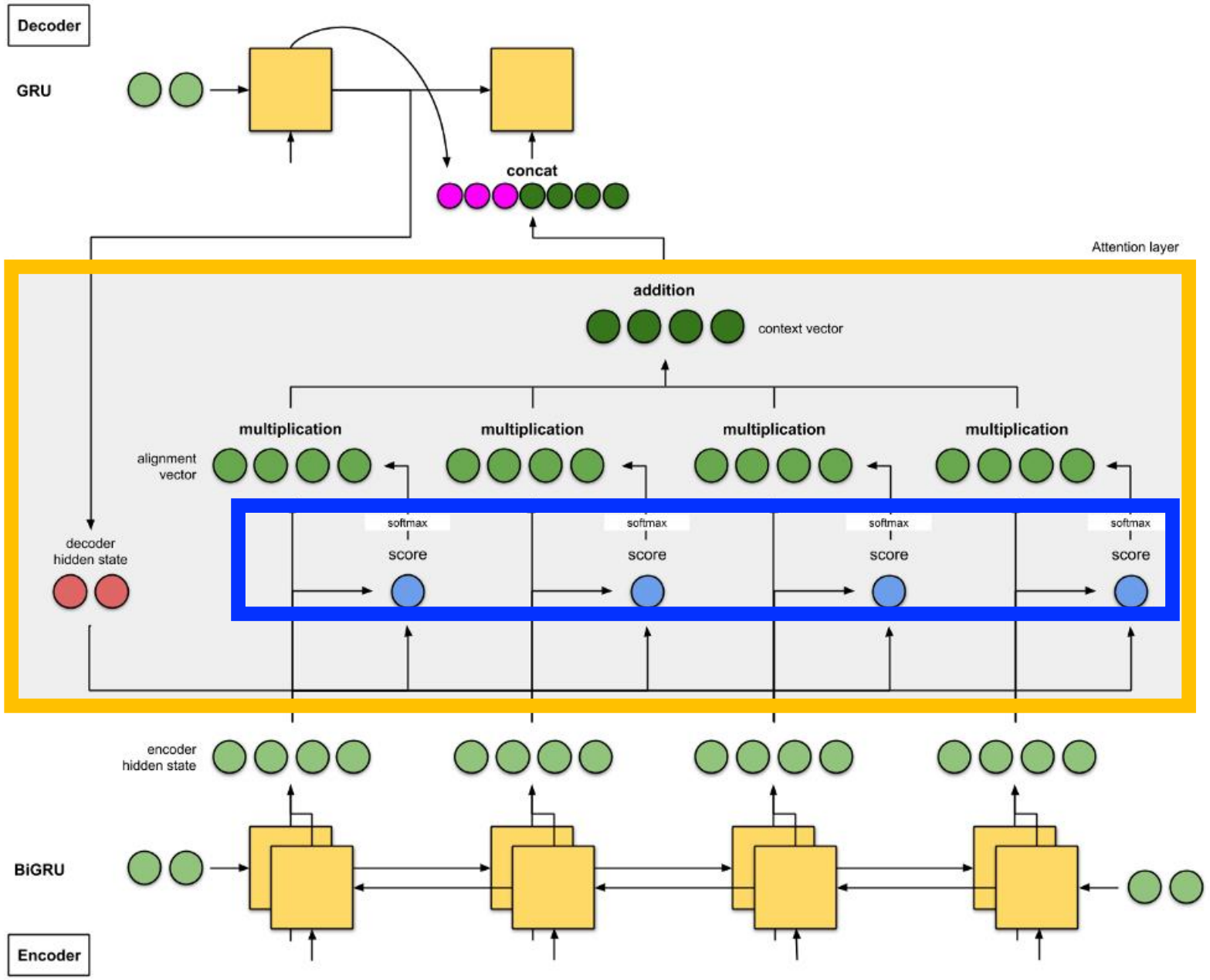
Then, softmax scores to quantify each input's relevance; e.g.,

= softmax([2, 0, 3, 3])

= [0.15, 0.02, 0.41, 0.41])

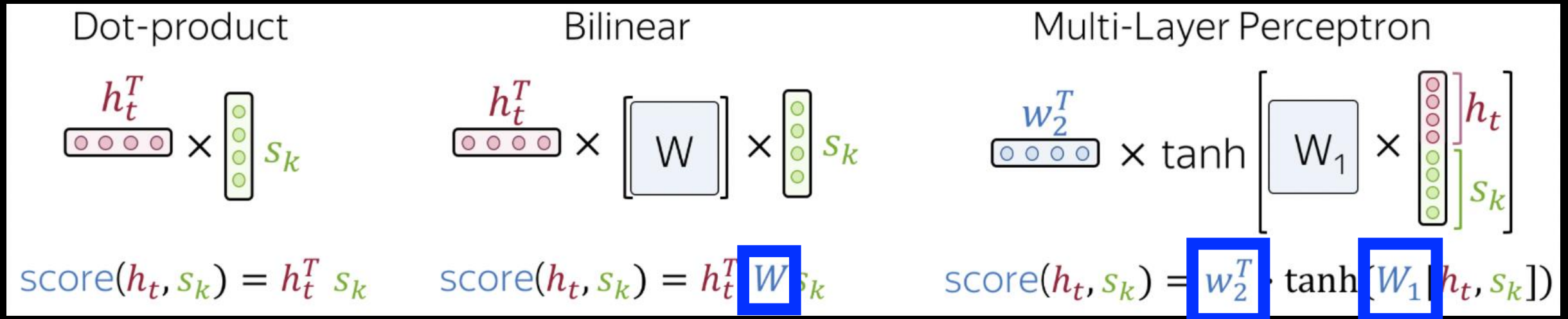
Which input(s) are most influential?

Which input(s) are least influential?



https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Measures: Many (Differentiable) Options



(no parameters)

What model parameters must be learned for each?

https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html

Attention Layer Summary

Attention output

$$c^{(t)} = a_1^{(t)} s_1 + a_2^{(t)} s_2 + \cdots + a_m^{(t)} s_m = \sum_{k=1}^{m} a_k^{(t)} s_k$$

"source context for decoder step t"

(weighted sum)

Attention weights

$$a_k^{(t)} = \frac{\exp(\text{score}(h_t, s_k))}{\sum_{i=1}^{m} \exp(\text{score}(h_t, s_i))}, \text{k} = 1..\text{m}$$

"attention weight for source token k at decoder step t"

(softmax)

Attention scores

$$\text{score}(h_t, s_k), \text{k} = 1..\text{m}$$

"How relevant is source token k for target step t?"

Attention input

$s_1, s_2, \ldots, s_m$ — all encoder states

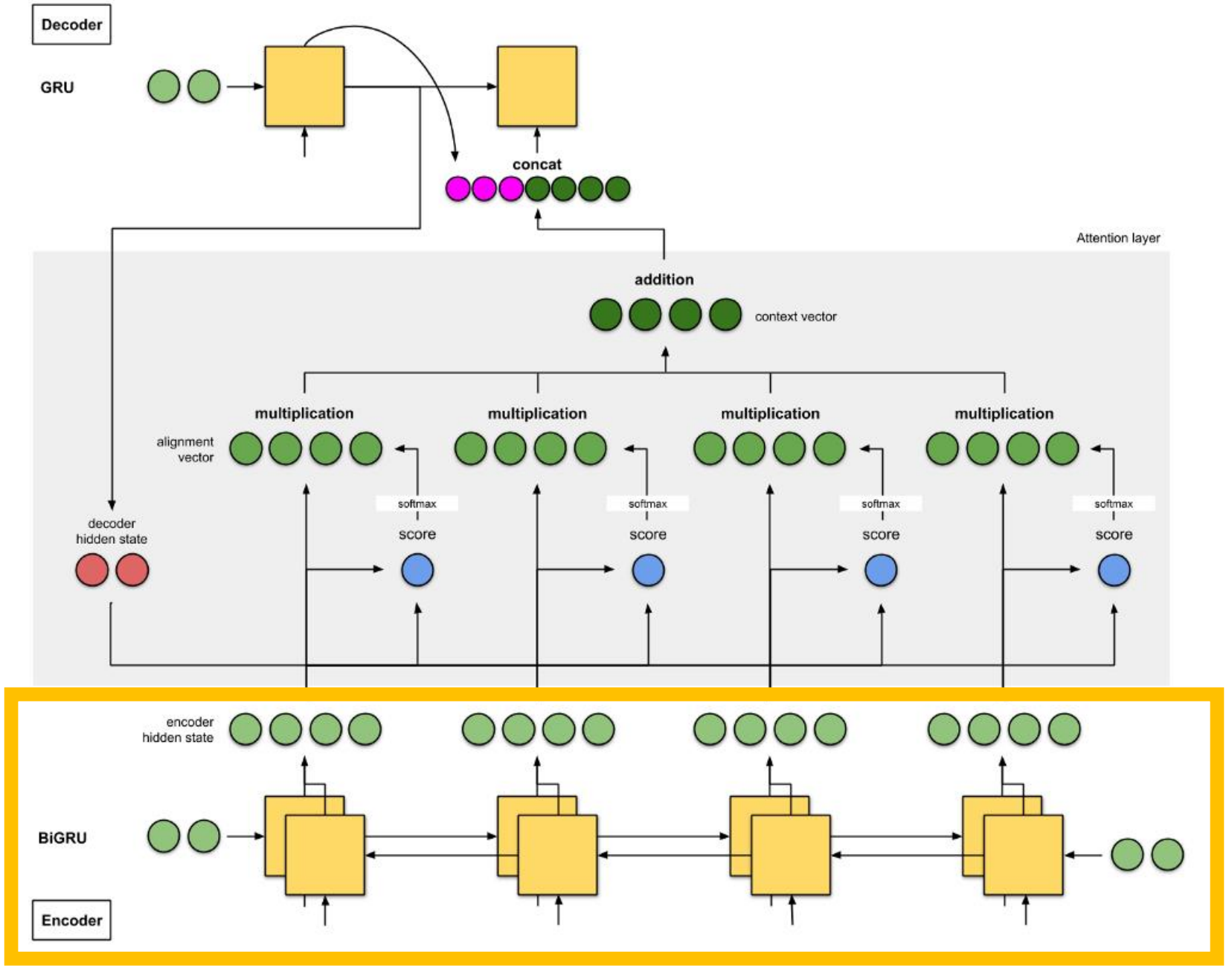
h_t — one decoder state

(all steps are differentiable)

https://lena-voita.github.io/nlp_course/seq2seq_and_attention.html

Encoder: Preparing the Input

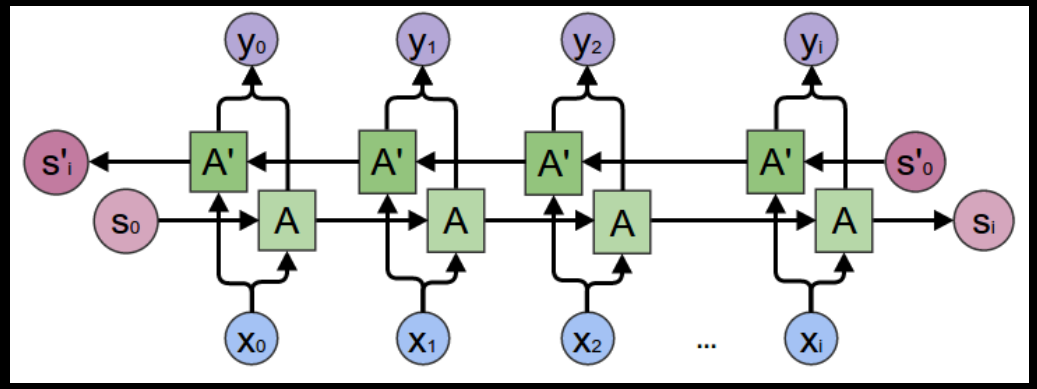
Converting input sequence to hidden state representations



https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Encoder: Preparing the Input

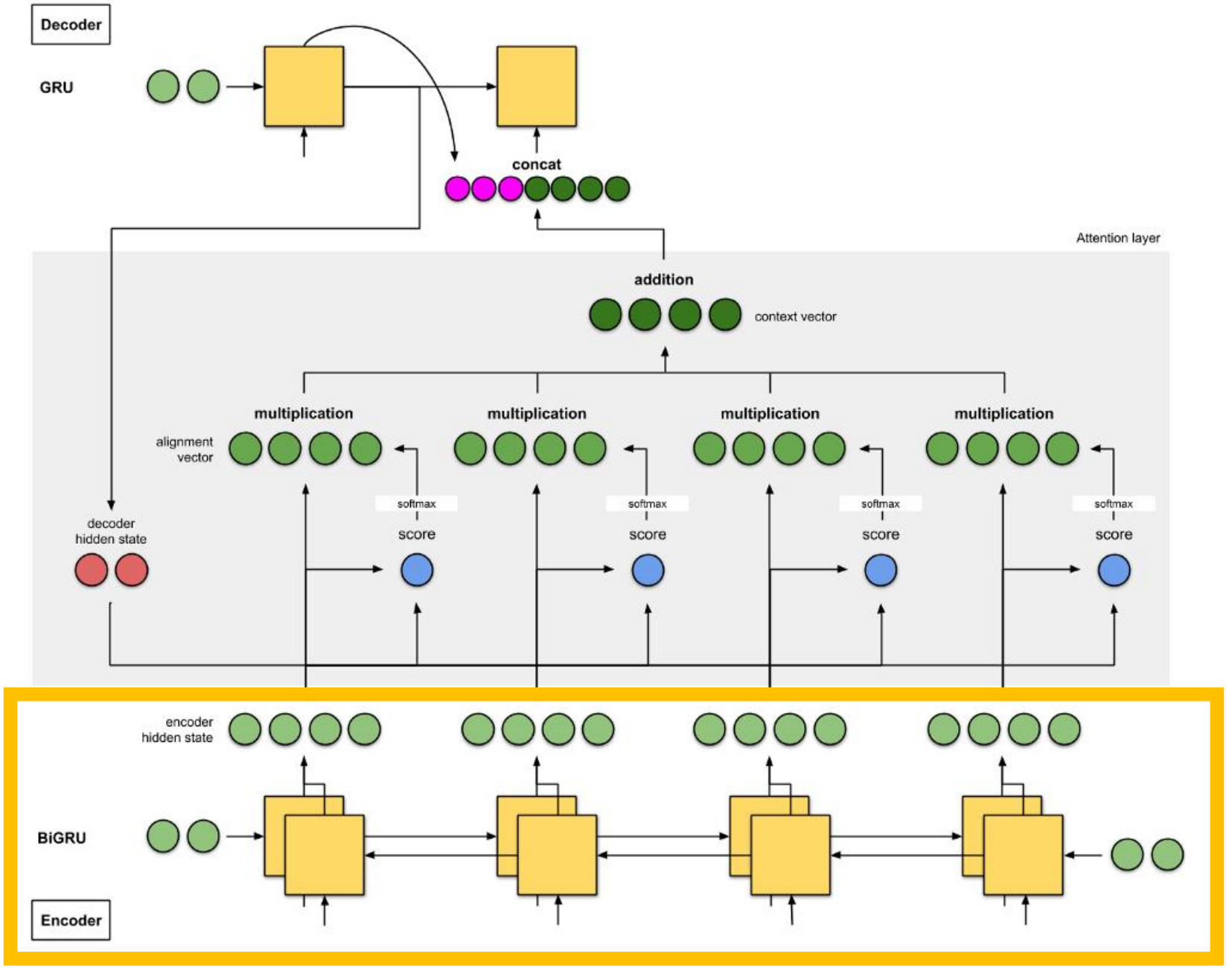
- Bidirectional GRUs (types of RNNs), where input is fed forward and backward and then their hidden states are concatenated into a final hidden state
  - Note: each hidden state tends to summarize only closer inputs



- What advantages do bi-directional RNNs have to uni-directional RNNs?
  - Information from the past and future contribute to predictions: e.g., can resolve for "Teddy is a …?" if Teddy refers to a "bear" or "former US President Roosevelt"
- What are disadvantages of bi-directional RNNs?
  - Entire sequence must be observed (e.g., unsuitable for text prediction)

https://towardsdatascience.com/understanding-bidirectional-rnn-in-pytorch-5bd25a5dd66

Encoder: Preparing the Input



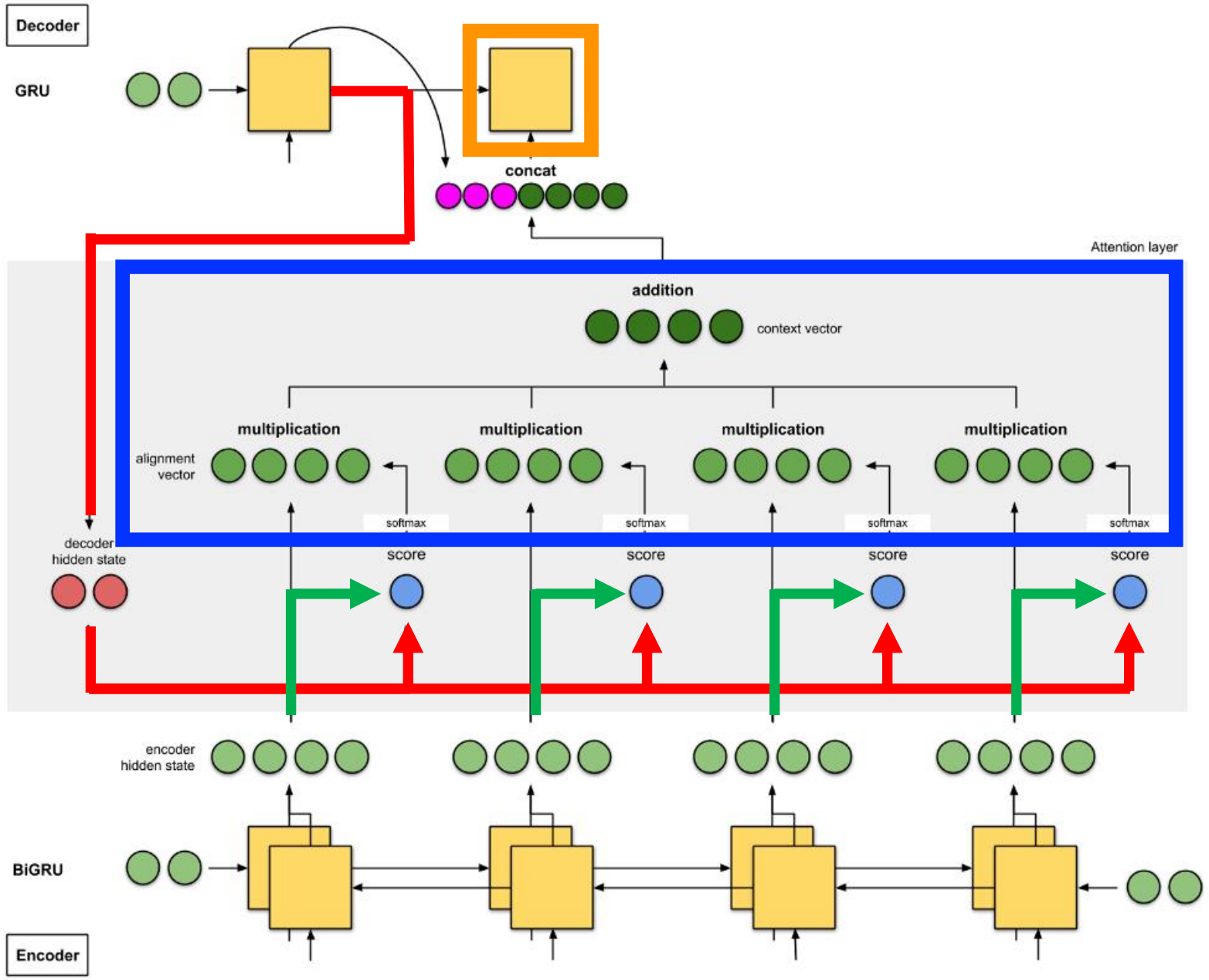
Note: includes embedding matrix to condense dimensionality of inputs to bi-directional RNN

https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Computational Analysis

What stays the same at each time step?
- input's hidden state

What changes at each time step?
- decoder's hidden state
- (and so) attention weights and context vector
- decoder's prediction at the previous time step

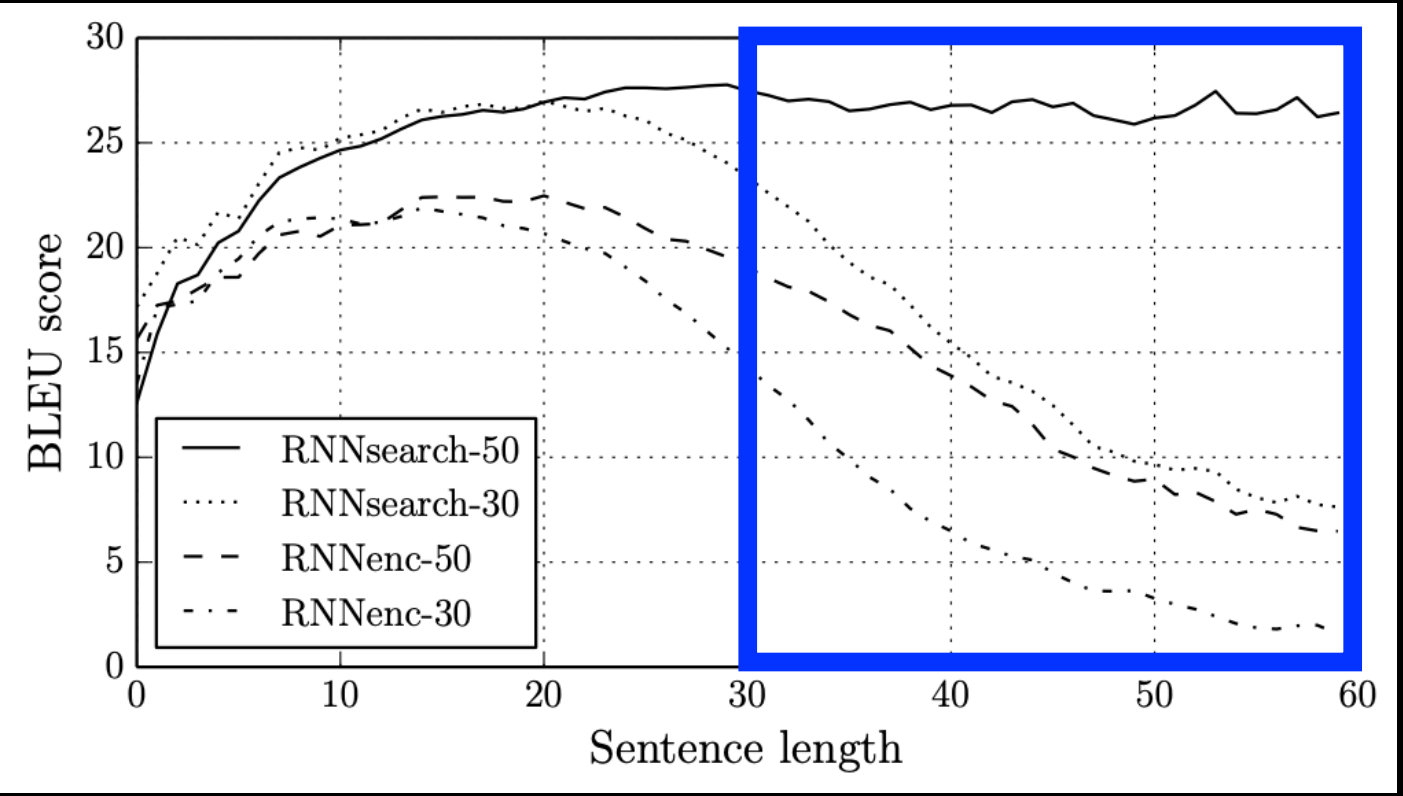


https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Training

- Dataset: English-to-French translation with 348M words
  - Uses 30,000 most frequent words from each language and [UNK] for the rest
- Weight initialization: Gaussian distributions with 0 mean and 0.01/0.001 variances
- Optimizer: adapts learning per parameter (Adadelta)
- Batch size: 80 sentences
- Decoding with beam search

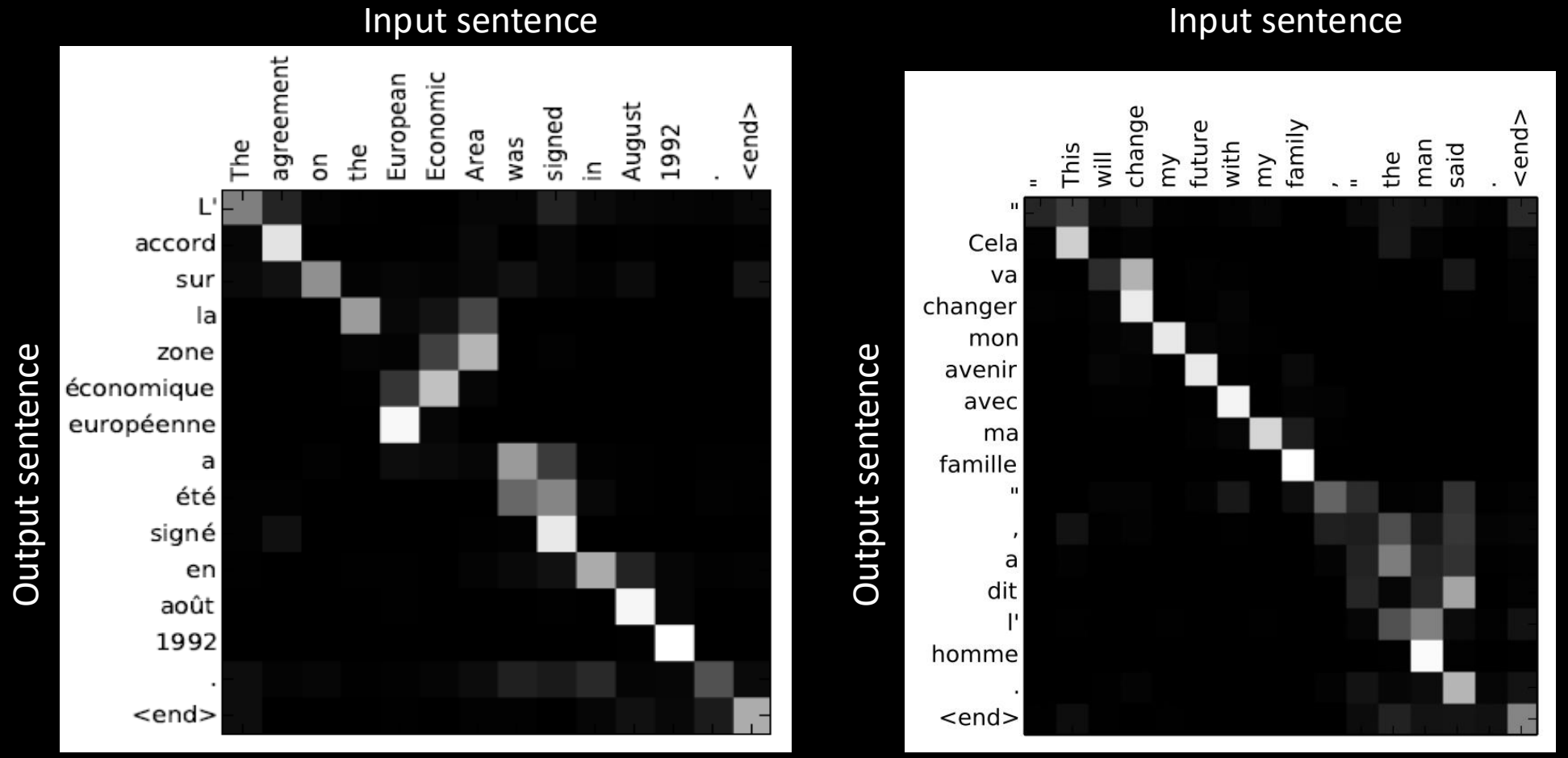
Experiments: Attention Improves Performance!



- Compared to encoder-decoder baseline, with sentences of lengths up to 30 and 50 respectively
- For which source sentence lengths does the model achieve the greatest performance gain over the baseline?

[Bahdanau, Cho, and Bengio]. ICLR 2015]

Visualizing Attention: Group Discussion

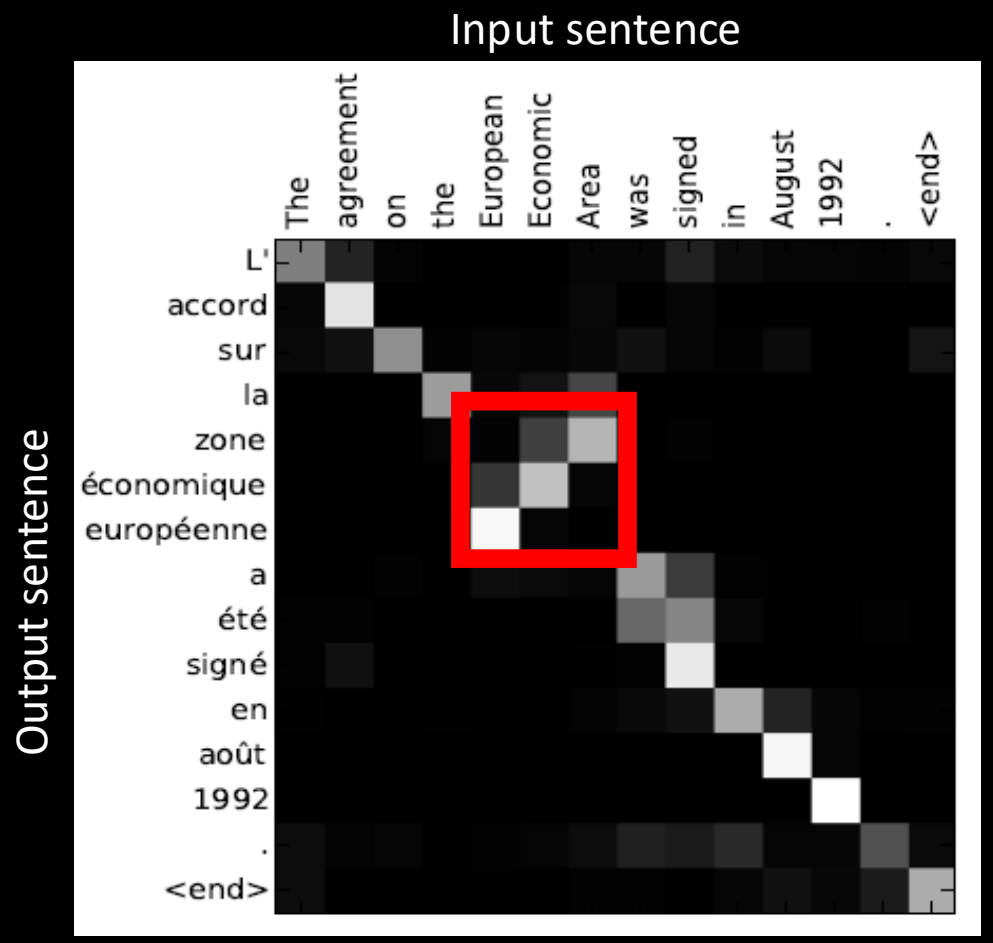


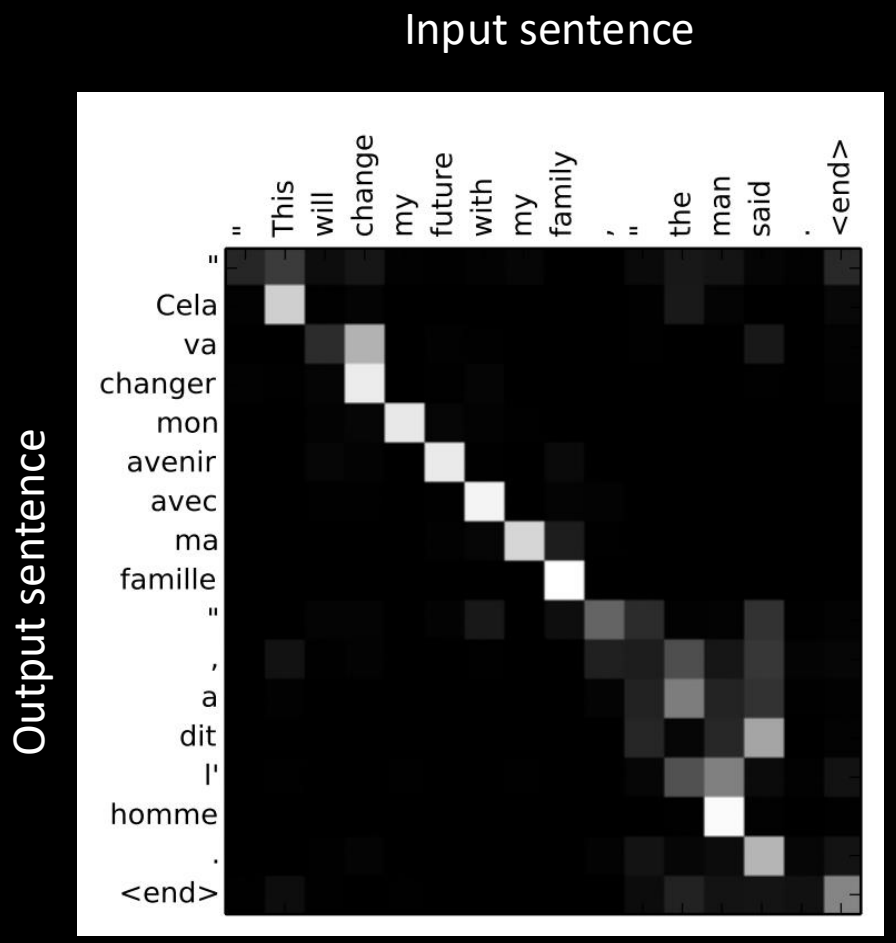
(Values are 0 to 1, with whiter pixels indicating larger attention weights)

What insights can we glean from these two examples?

[Bahdanau, Cho, and Bengio]. ICLR 2015]

Visualizing Attention: Interpretable

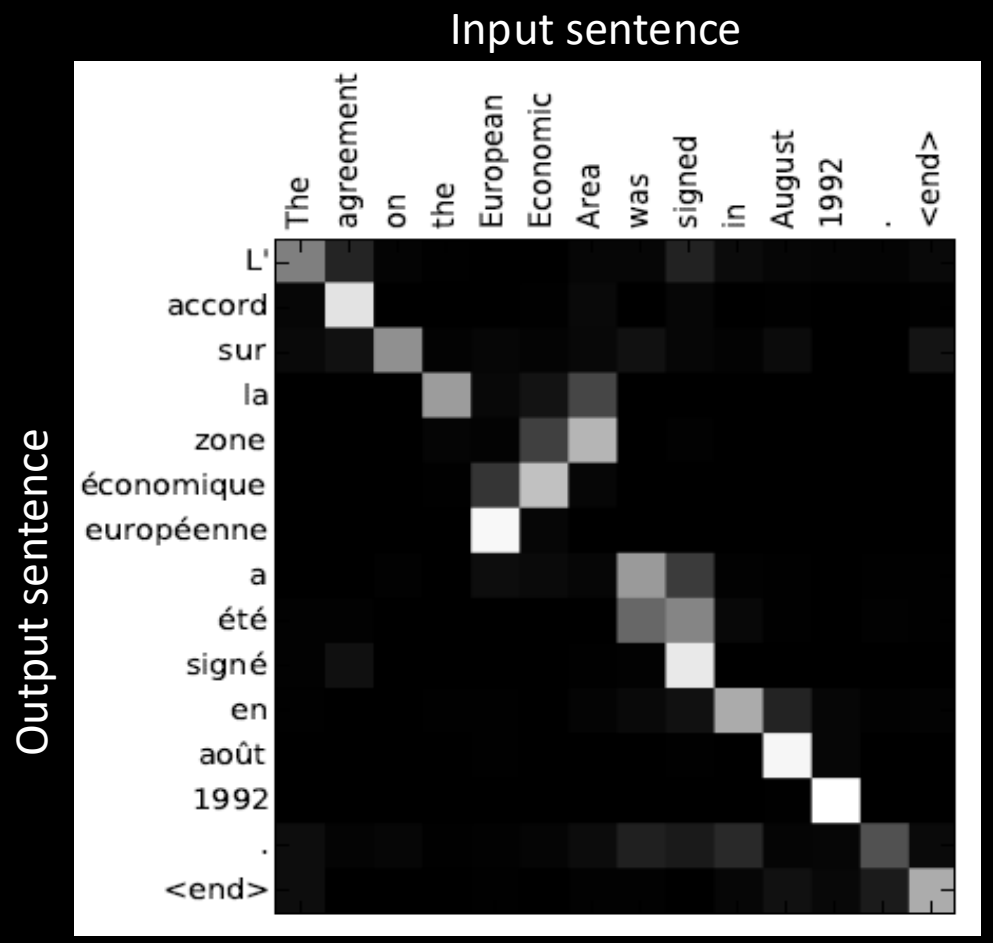


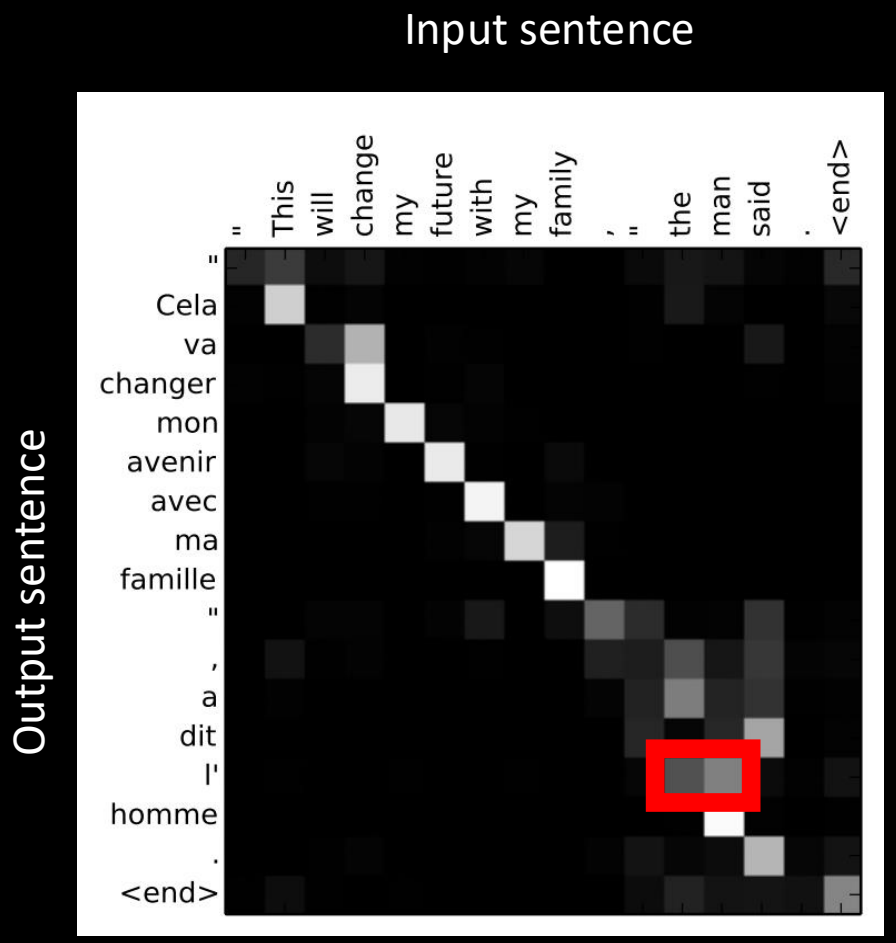


While a linear alignment between input and output sentences is common, there are exceptions (e.g., order of adjectives and nouns can differ)

[Bahdanau, Cho, and Bengio]. ICLR 2015]

Visualizing Attention: Interpretable

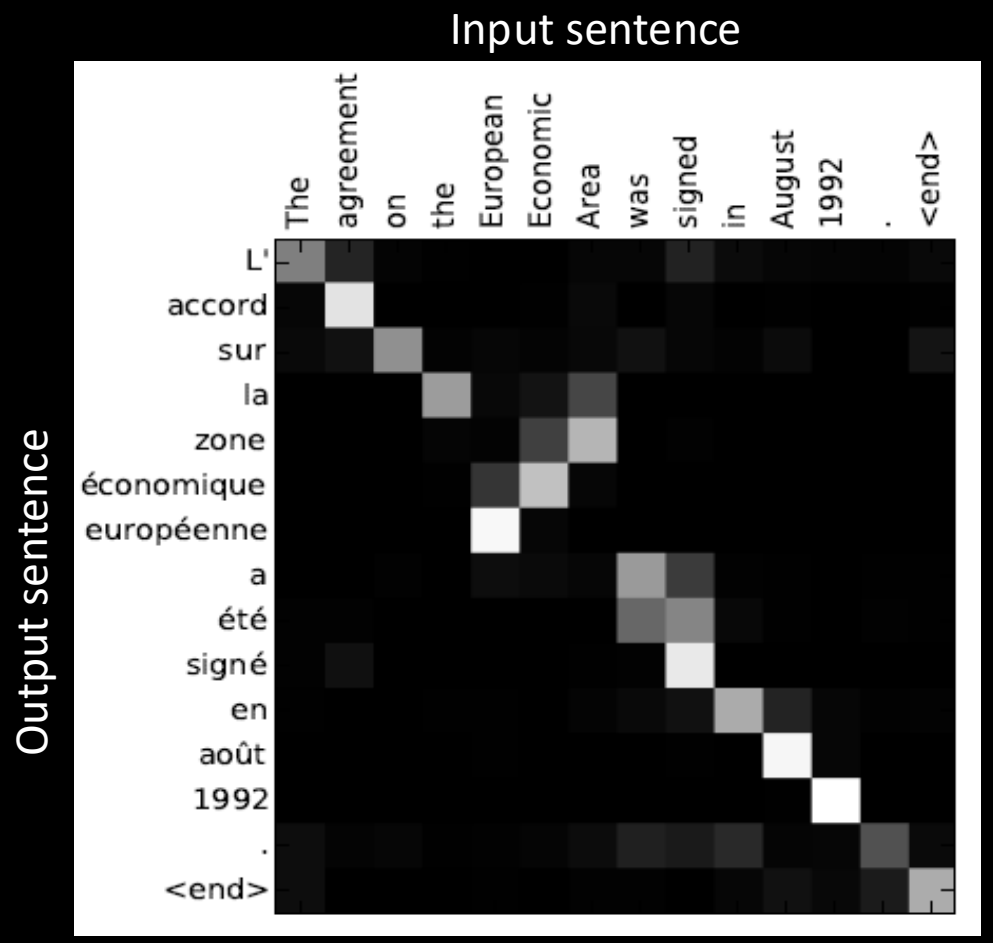


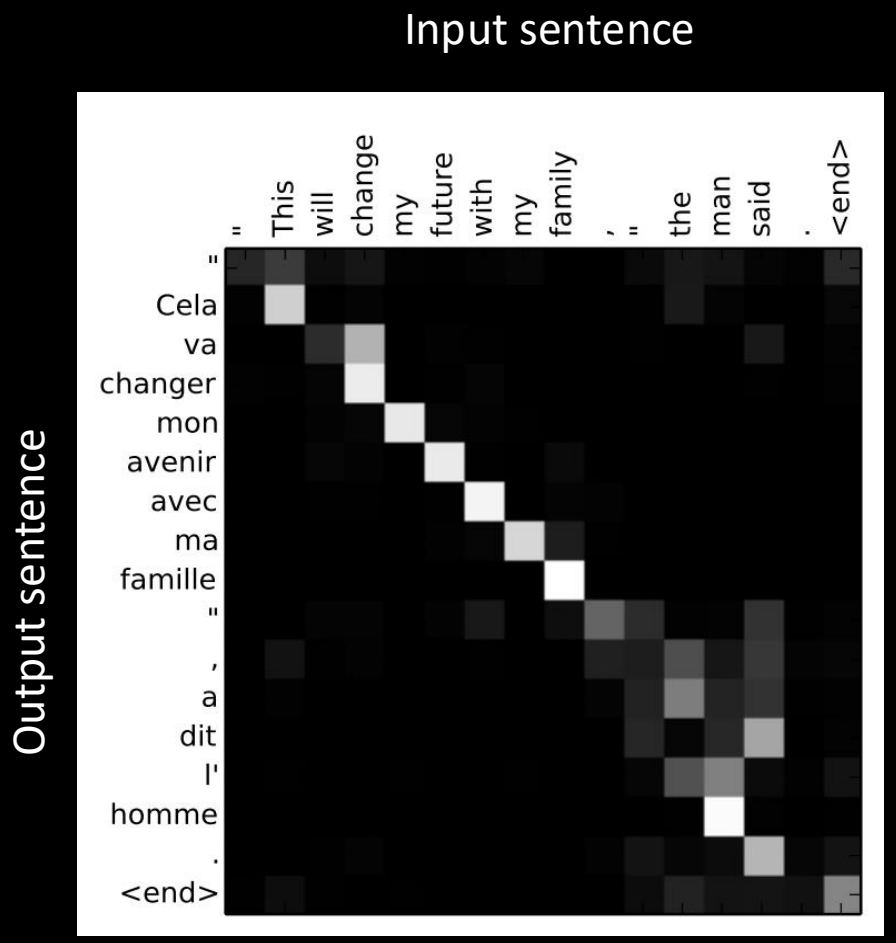


Output words are often informed by more than one input word;
e.g., “man” indicates translation of “the” to l’ instead of le, la, or les

[Bahdanau, Cho, and Bengio]. ICLR 2015]

Visualizing Attention: Interpretable





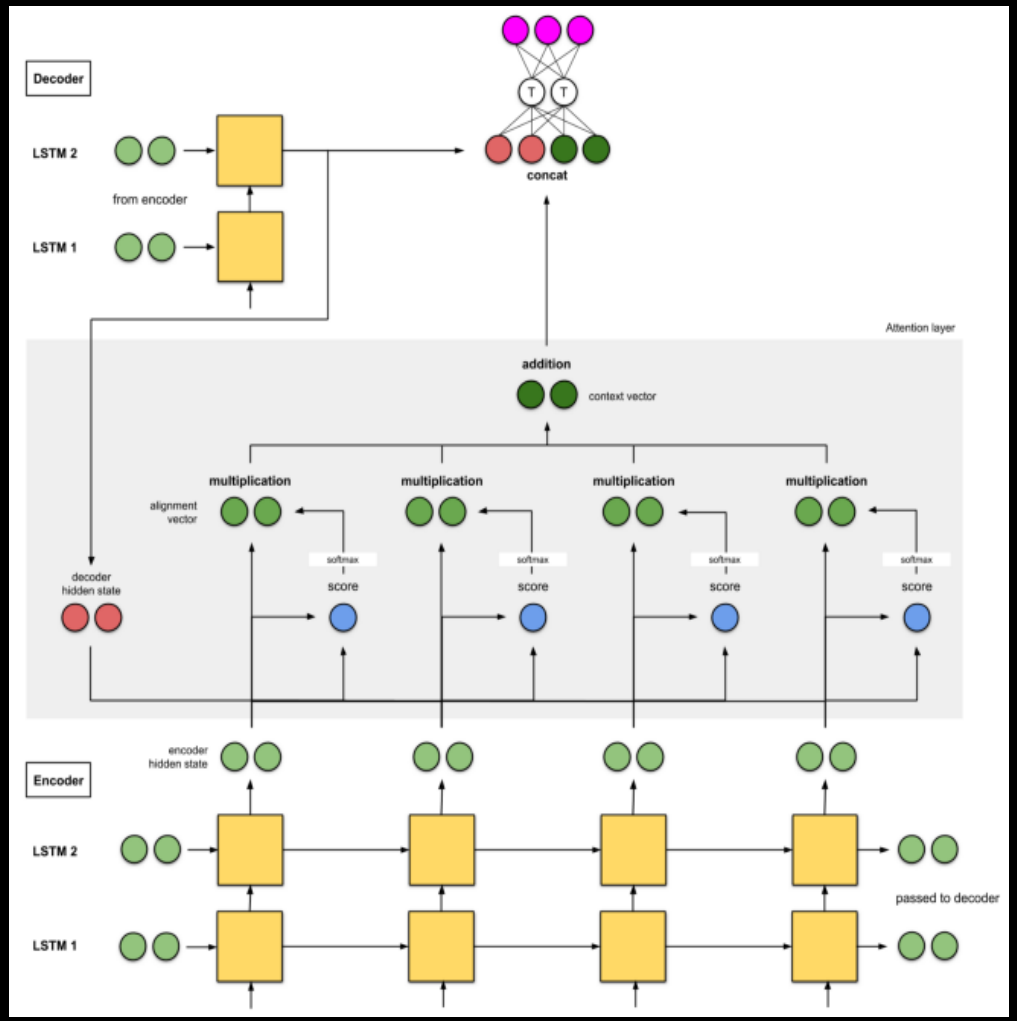
The model accounts for different input and output lengths
(e.g., 1 extra output word for both examples)

[Bahdanau, Cho, and Bengio]. ICLR 2015]

Many Variants Followed

(Luong et al, EMNLP 2015)

Introduces feedforward neural network

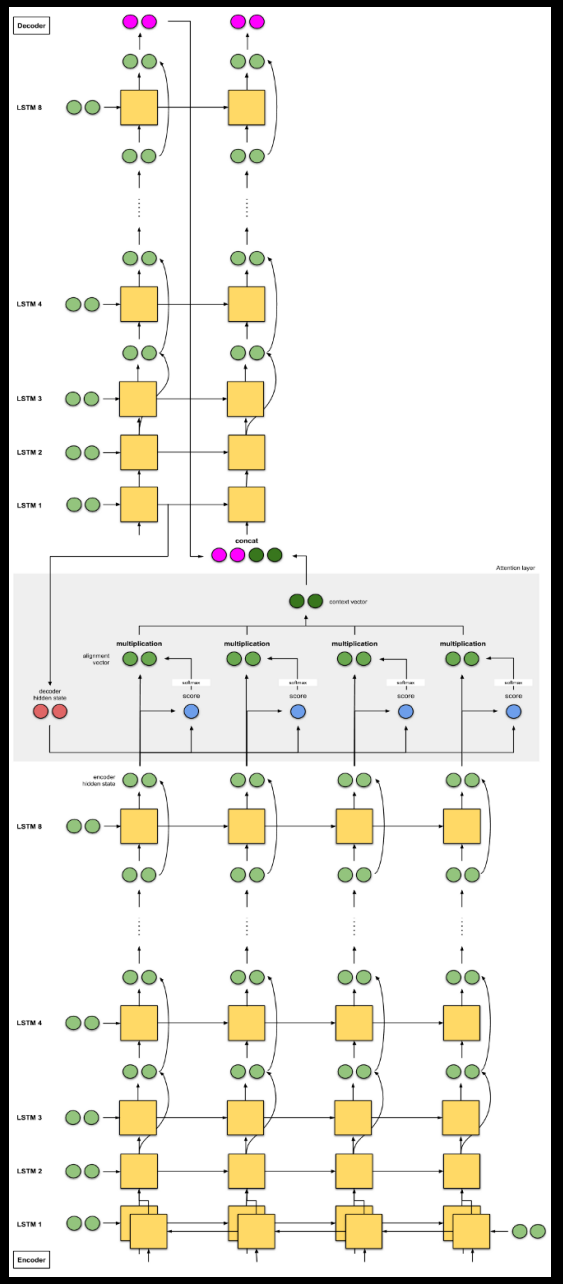


2-layer stacked LSTM

(Wu et al., arXiv 2016)

(Google introduced and used this for a production machine translation service)

Introduces MANY hidden layers



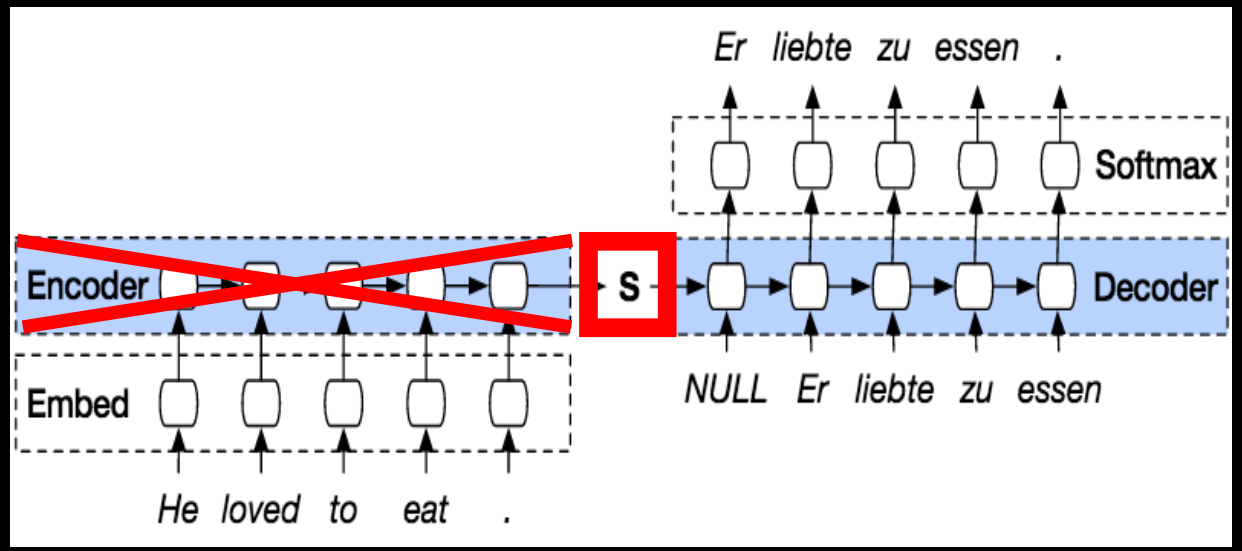
8 layers: more layers and skip connections between layers

(greater level of abstraction for input)

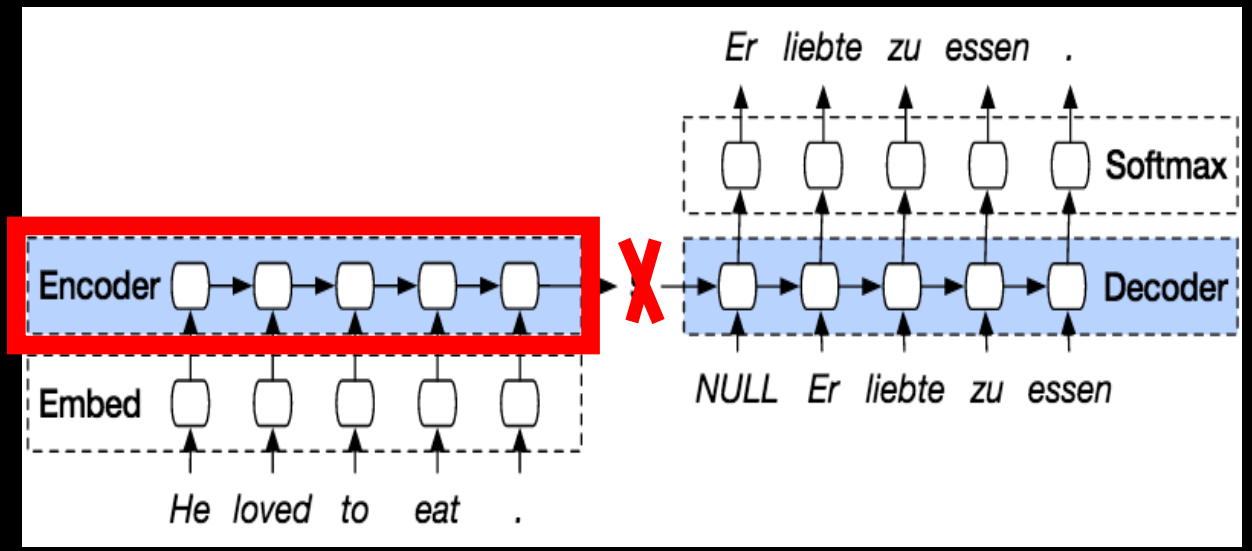
https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Summary: RNN Encoding vs Attention

Decoder only sees summary of input



Decoder directly attends to (weighted) inputs



"One of challenges left for the future is to better handle unknown, or rare words."

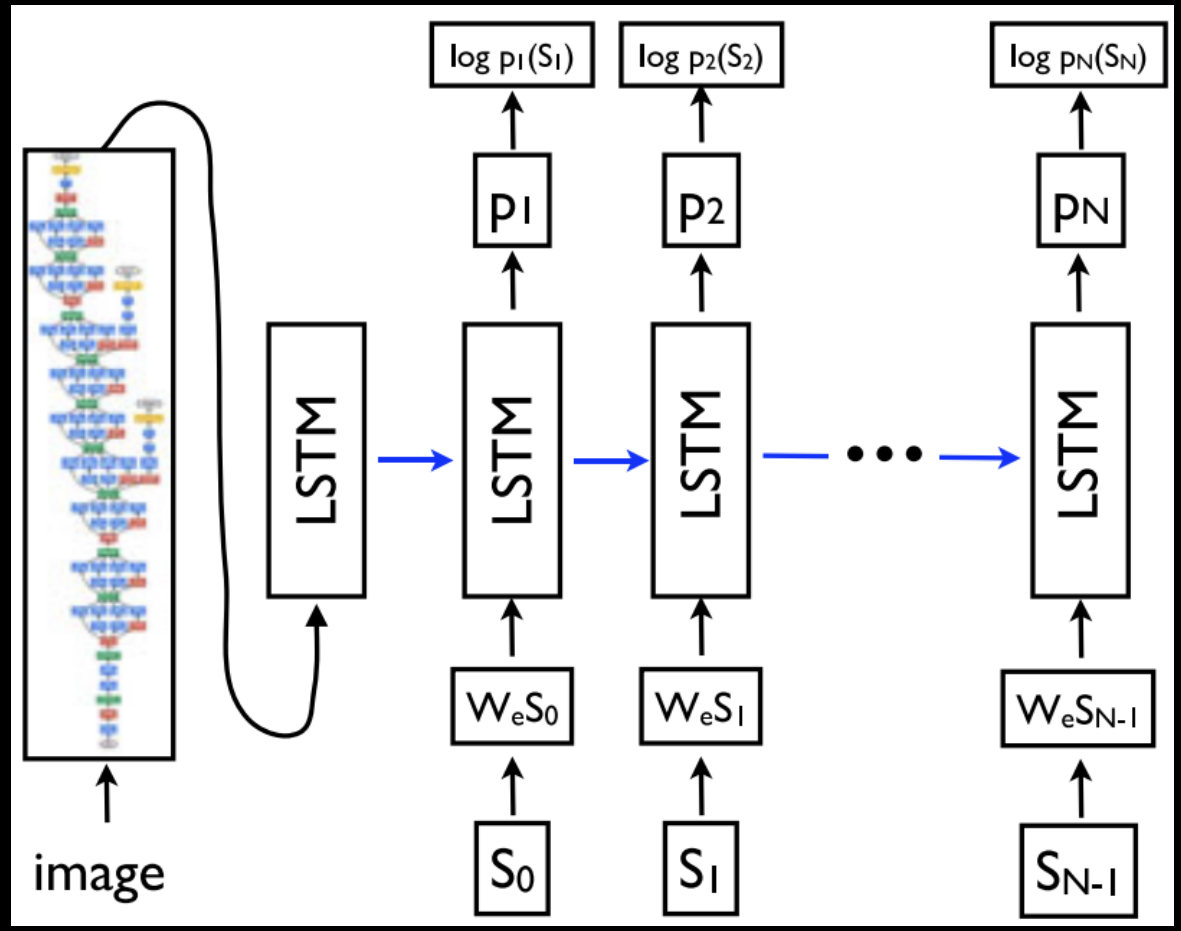
[Bahdanau, Cho, and Bengio]. ICLR 2015]

Image source: https://smerity.com/articles/2016/google_nmt_arch.html

Today's Topics

- Mystery: why did neural networks struggle to translate long sentences?
- Idea: instead of summarizing everything, attend to relevant inputs
- Beyond text: attention for image captioning
- Beyond vision-language problems: attention for speech recognition

Recall Captioning Model: Fixed Length Vector Summarizing Input Image Fed to Decoder



Vinyals et al. Show and Tell: A Neural Image Caption Generator. CVPR 2015.

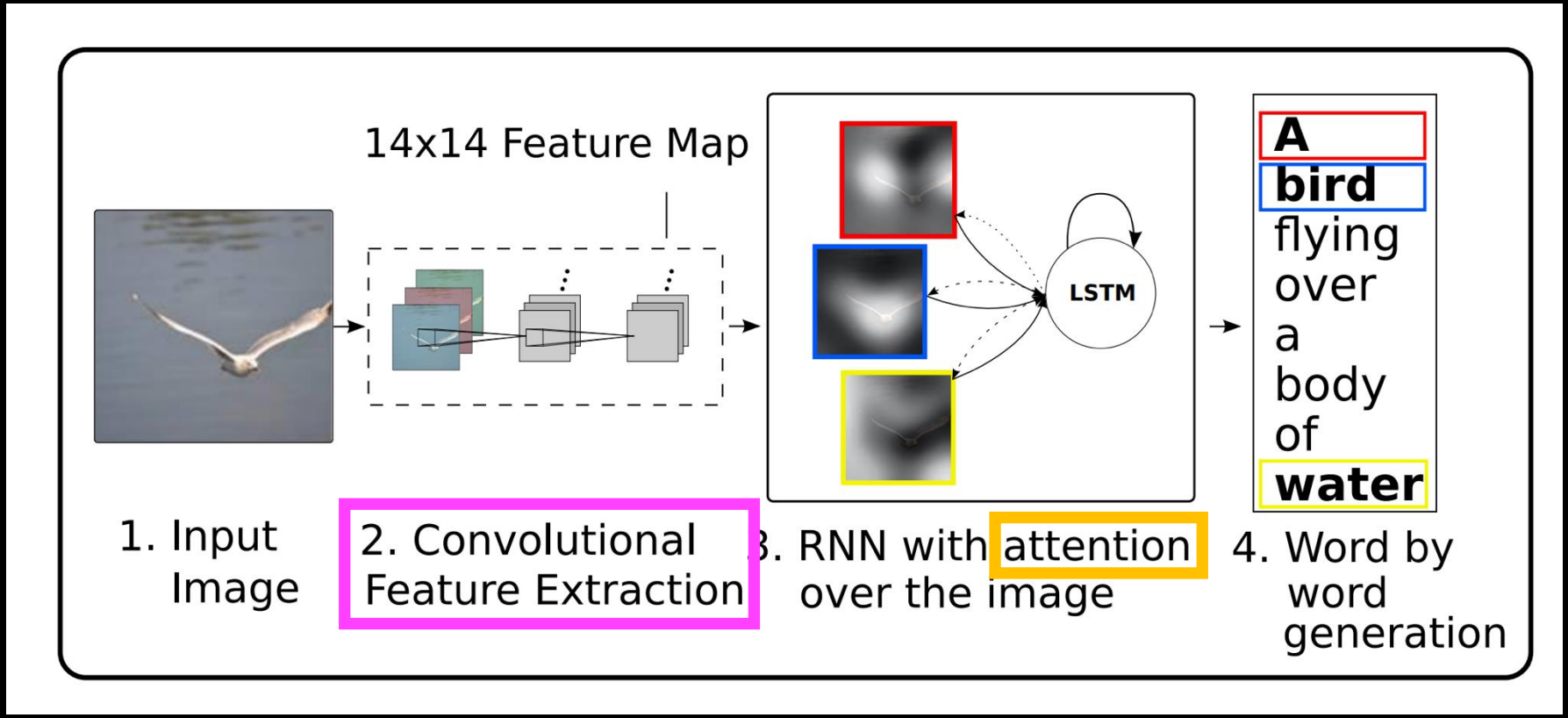
(ICML, 2015)

Show, Attend and Tell: Neural Image Caption Generation with Visual Attention

Kelvin Xu KELVIN.XU@UMONTREAL.CA
Jimmy Lei Ba JIMMY@PSI.UTORONTO.CA
Ryan Kiros RKIROS@CS.TORONTO.EDU
Kyunghyun Cho KYUNGHYUN.CHO@UMONTREAL.CA
Aaron Courville AARON.COURVILLE@UMONTREAL.CA
Ruslan Salakhutdinov RSALAKHU@CS.TORONTO.EDU
Richard S. Zemel ZEMEL@CS.TORONTO.EDU
Yoshua Bengio FIND-ME@THE.WEB

"Encouraged by recent advances in caption generation and inspired by recent success in employing attention in machine translation (Bahdanau et al., 2014) and object recognition (Ba et al., 2014; Mnih et al., 2014), we investigate models that can attend to salient part of an image while generating its caption."

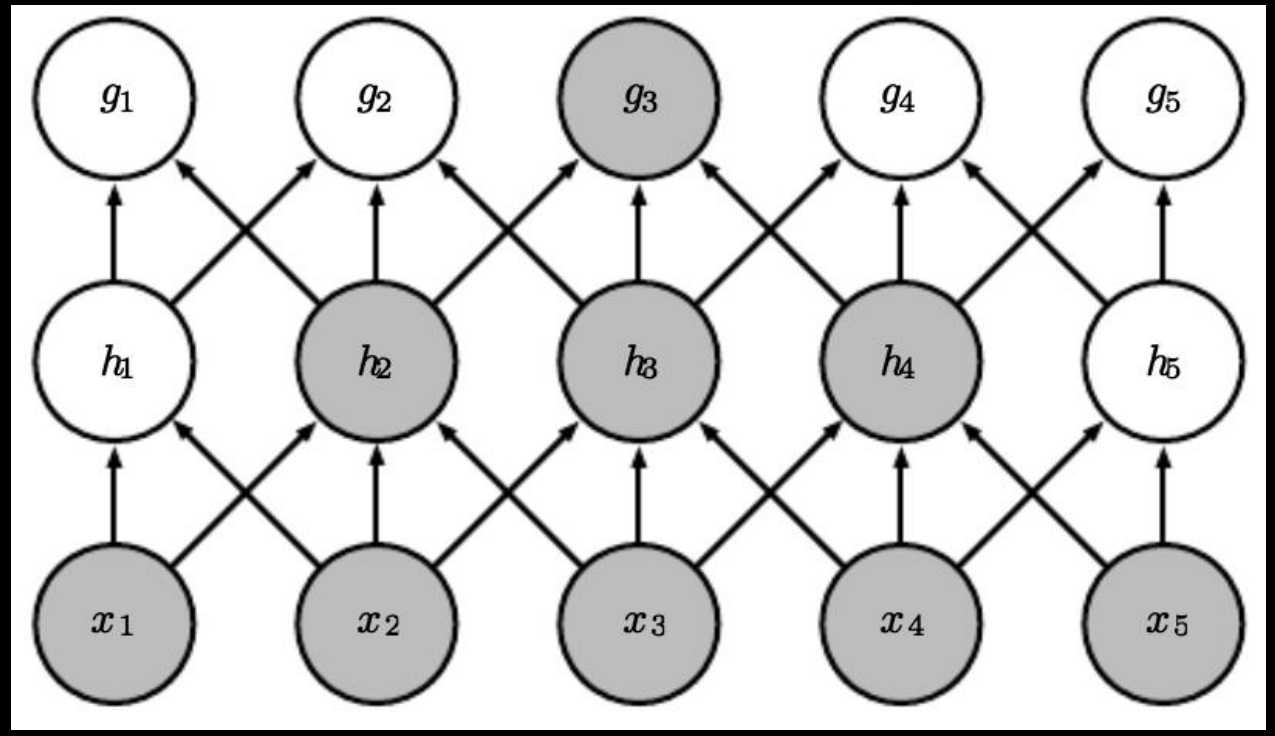
Idea: Decoder Selects Relevant Input Regions



Attention weights operate on feature vectors corresponding to coarse image regions (coming from stacked convolution layers)

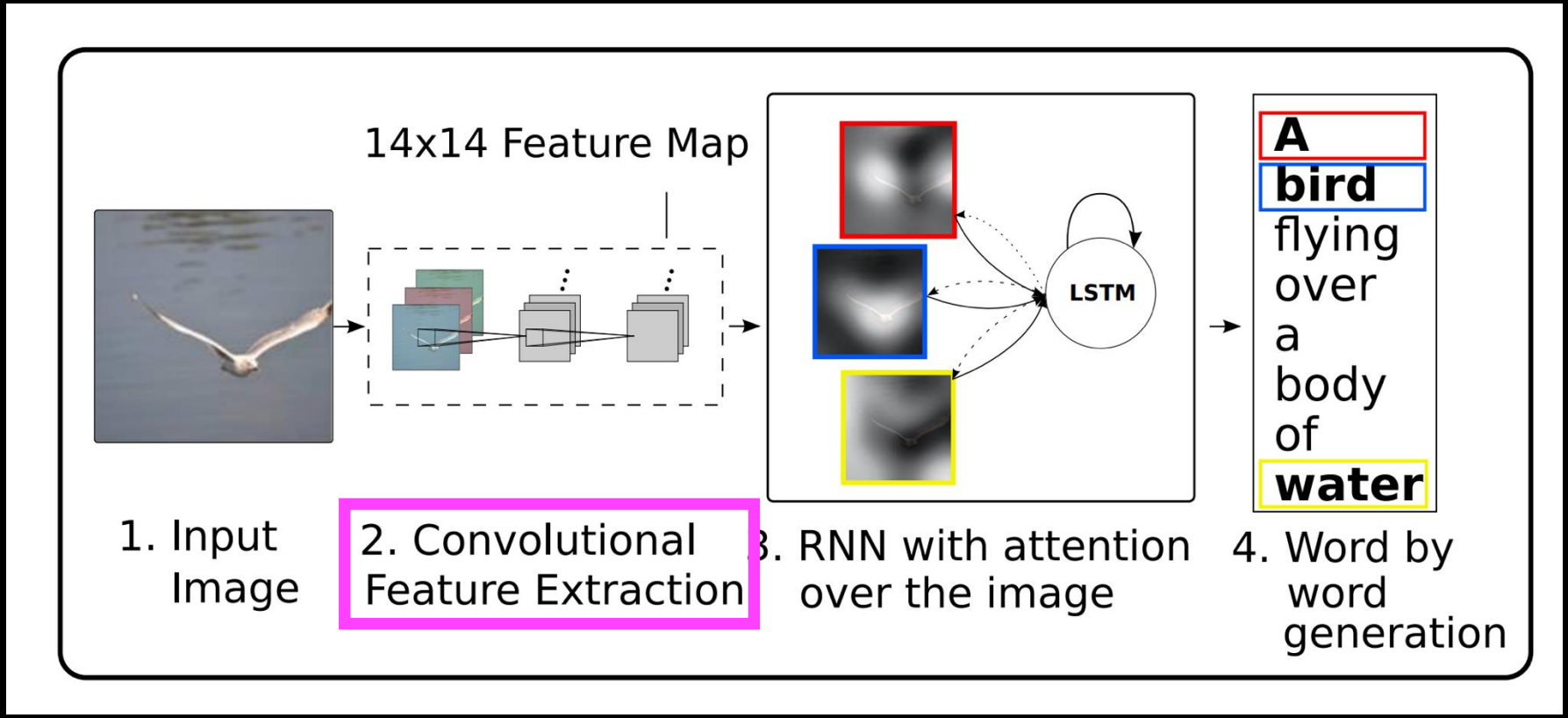
Xu et al. ICML 2015.

Recall: Coarse Image Regions Emerge from Stacked Convolutional Layers



https://www.deeplearningbook.org/contents/convnets.html

Implementation: Trained Similar to Pioneering Paper for Nearly 3 Days (on Titan Black GPU)



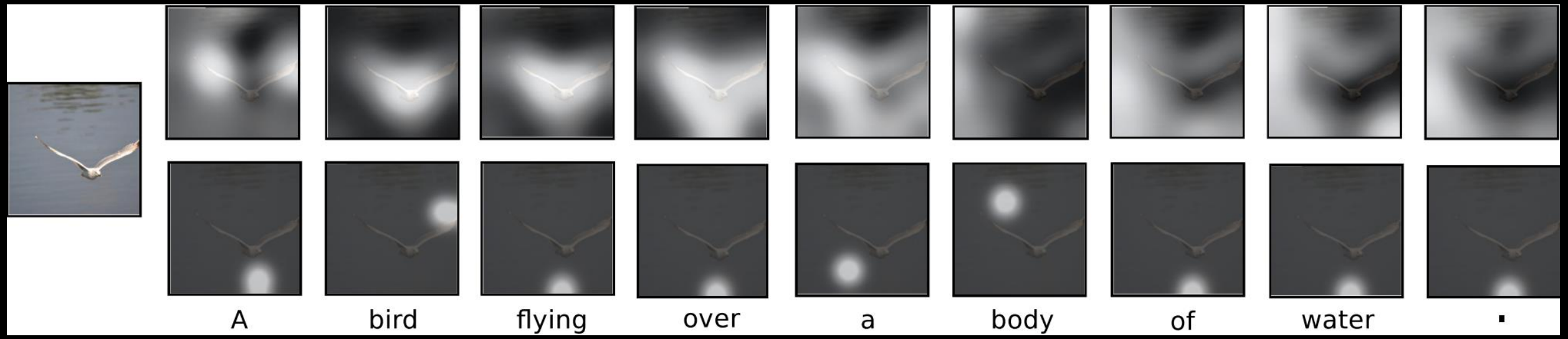
Features: extracted from 4th convolutional layer of 2014 ImageNet challenge winner, VGG

Vocabulary size: 10,000

Xu et al. ICML 2015.

Achieved state-of-the-art performance on three benchmark datasets
(Flickr8k, Flickr30k and MS COCO)

Interpretability: Visualizing Attention



Xu et al. ICML 2015.

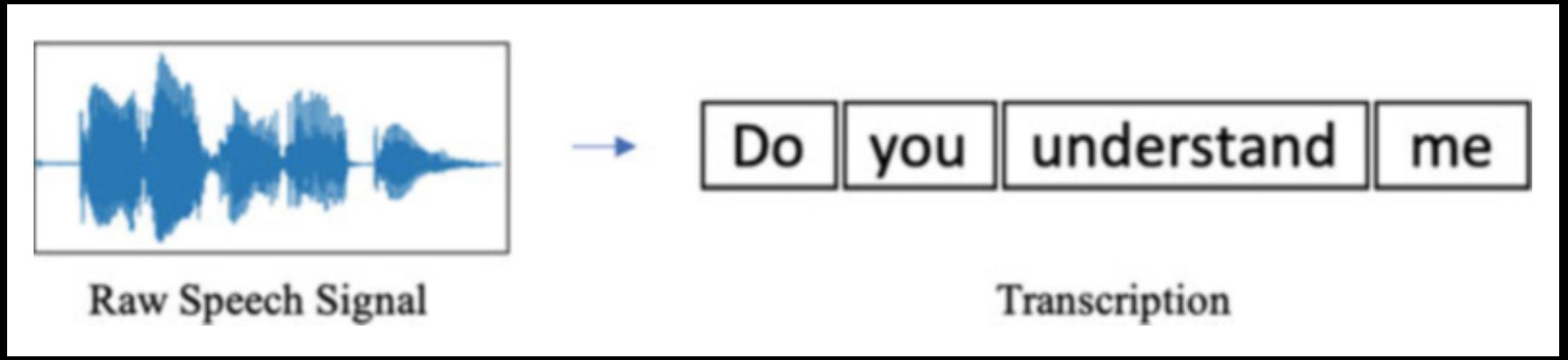
Today's Topics

- Mystery: why did neural networks struggle to translate long sentences?
- Idea: instead of summarizing everything, attend to relevant inputs
- Beyond text: attention for image captioning
- Beyond vision-language problems: attention for speech recognition

Speech Recognition: Task Definition

Input: spoken language

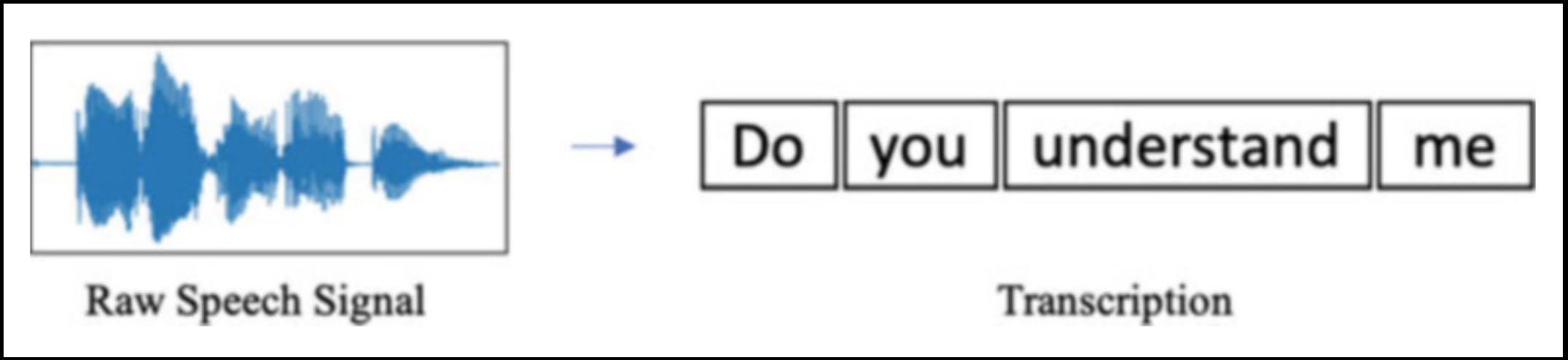
Output: machine readable text



(Compression waves from pushing air from one's lungs and modulating it using one's tongue, teeth, and lips)

Kamath, Liu, and Whitaker. Deep Learning for NLP and Speech Recognition. 2019.

Speech Recognition: Task Difficulties



- Diverse speakers: accents, volumes, pace, and cadence
- Diverse recording quality
- Background noise

- Segmenting signal into distinct words

Kamath, Liu, and Whitaker. Deep Learning for NLP and Speech Recognition. 2019.

(ICASSP, 2016)

LISTEN, ATTEND AND SPELL: A NEURAL NETWORK FOR LARGE VOCABULARY CONVERSATIONAL SPEECH RECOGNITION

William Chan

Carnegie Mellon University

Navdeep Jaitly, Quoc Le, Oriol Vinyals

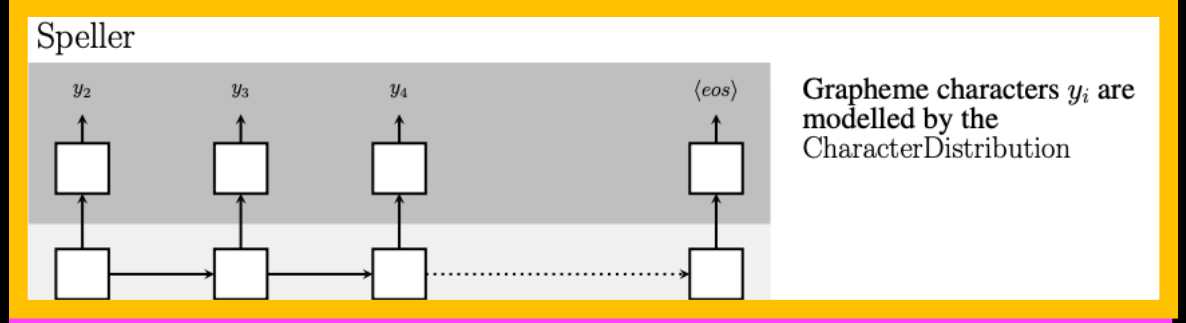
Google Brain

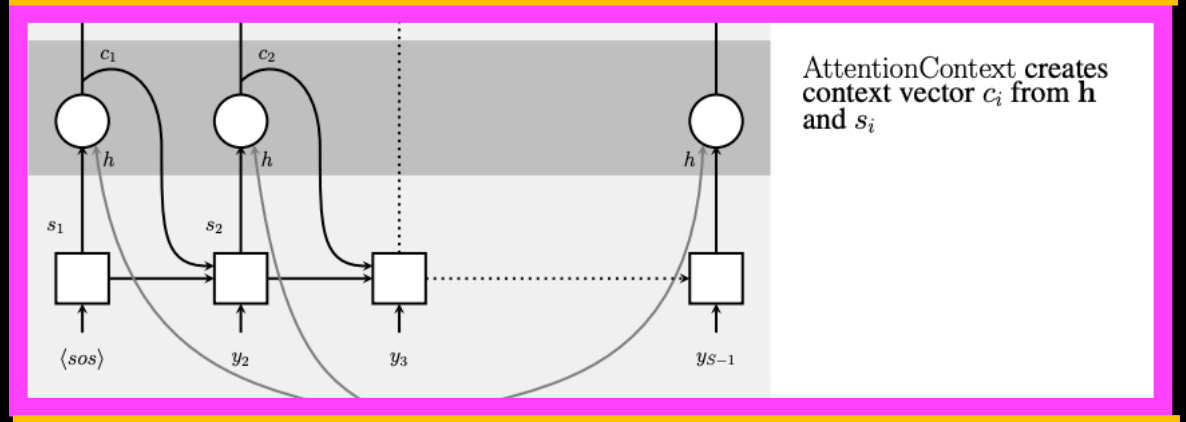
Architecture

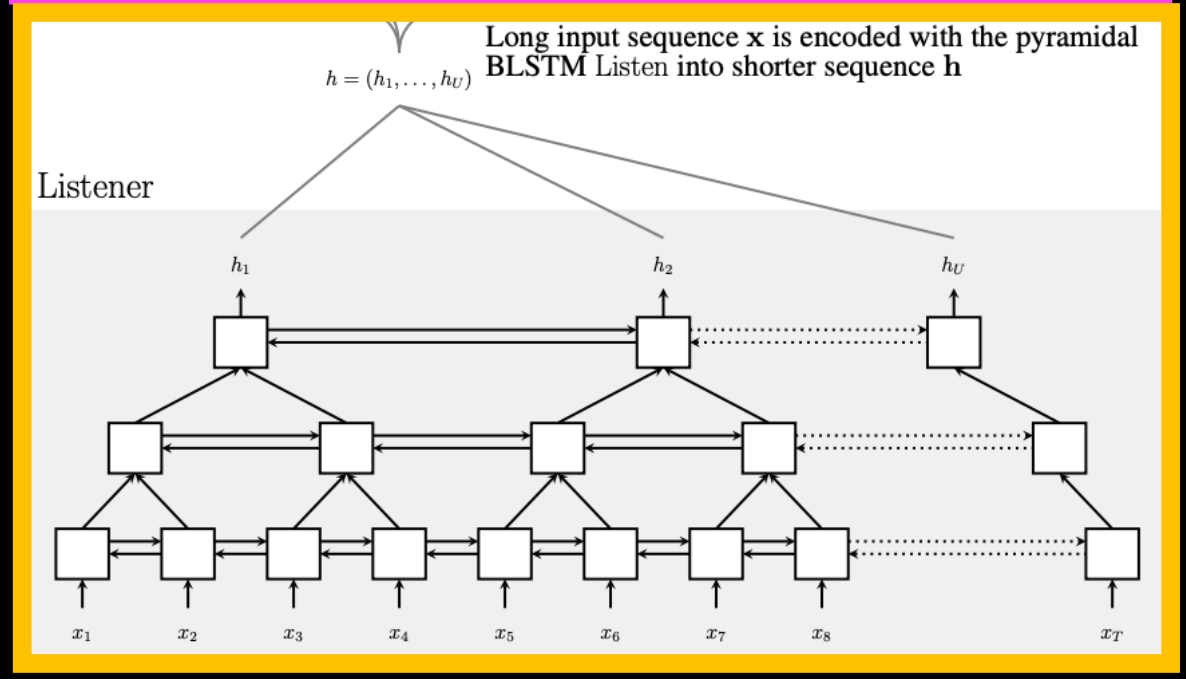
Decoder: at each time step, 2-layer LSTM used to predict a character

Attention module: determines each input's relevance for prediction

Encoder: converts feature vector sampled from the input (40-d log-mel filter bank features computed every 10ms) into hidden states; much longer speech sequences necessitated innovation (e.g., pyramid) to compress input







Training (Took ~2 Weeks)

- Dataset: ~3M Google voice search examples/utterances (~2000 hours of data)
- Weight initialization: Uniform distribution from -0.1 to 0.1
- Optimizer: SGD with learning weight decay throughout training
- Batch size: 32 utterances
- Data augmentation: 20x increase by introducing various noise sources
- Decoding with beam search: beam width of 32

Experiments: Close to State-of-the-Art Using Just a Single, End-to-End Trained Model

- Evaluation metric: edit distance between the prediction and the target as follows:

word replaced (e.g., “goose” transcribed as “choose”)

word added (e.g., “caboose” transcribed as “the goose”)

word omitted (e.g., “it is a fun” transcribed as “it is fun”)

$$\frac{\text{Substitutions + Insertions + Deletions}}{\text{Number of Words Spoken}}$$

- What indicates better performance: larger or smaller values?

Word Error Rate: Example

- Correct: The sun makes it look like uh a warm, day to go outside to adventure.
- Predicted: The son makes it to bike with a swarm to go outside to Denver today.
- Number of words spoken?
  - 15
- WER?

$$\frac{\text{Substitutions + Insertions + Deletions}}{\text{Number of Words Spoken}}$$

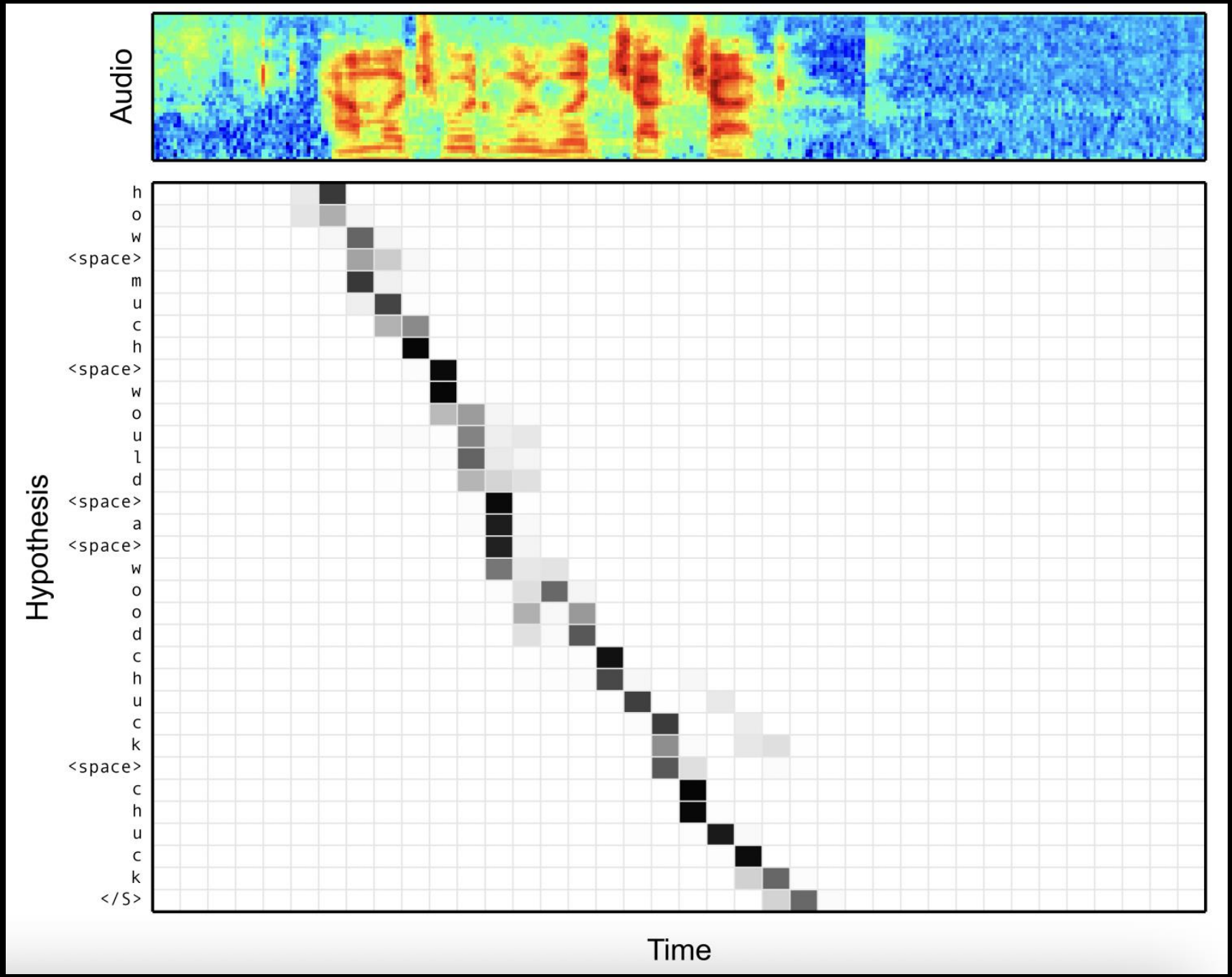
$$\frac{6 + 1 + 1}{15} = 0.53$$

Word Error Rate: Limitations

- Does not indicate why errors occur
  - Background noise (e.g., music, other talking)
  - Specialized language (i.e., words reflecting domain expertise)
  - Speaker pronunciations/accent
- Does not reflect whether transcription correctly captures:
  - Capitalization
  - Punctuation
  - Numbers
  - Paragraphs
- May indicate poor quality when humans could understand the content
- Weights all word errors equally

Visualizing Attention

Alignment between audio signal and characters:



Chan et al. Listen, Attend and Spell. ICASSP 2016.

Today's Topics

- Mystery: why did neural networks struggle to translate long sentences?
- Idea: instead of summarizing everything, attend to relevant inputs
- Beyond text: attention for image captioning
- Beyond vision-language problems: attention for speech recognition

The End